MANCHESTER 1824
The University of Manchester

AERO62520 - Robotic Systems Design Project: Final Design Review

Year of submission

2026

Team 5

School of Engineering

Contents

Word count: 19064

1 Introduction

1.1 Problem Statement and Objectives

Autonomous mobile robots are essential to optimise resource management and operational efficiency in the modern engineering industry. This project is directly motivated by the need to develop a robust, integrated autonomous platform to address difficulties in automated sorting and retrieval.

The project aims for the robot to operate autonomously to search and grasp target coloured objects in the test environment, then retrieve them and place them in colour-matched bins.

Core objectives include tasks as follows:

- Perform autonomous mapping and navigation in the test environment without hitting any obstacles.
- Detect three different coloured target objects and bins using the vision system.
- For each coloured object, approach it, grasp it using the manipulator, and place it into its colour-matched bin.
- Complete the entire mission within 20 minutes while maintaining accurate and robust performance under dynamic, real-time conditions.

1.2 Addressed feedback from the PDR

1.2.1 System Block Diagram Clarification

Feedback from the PDR indicated that the system block diagram was complete, but lacked a legend explaining the colour coding used to distinguish hardware categories. The component descriptions and justifications were already sufficient, so the main issue was clarity of presentation rather than the underlying system design.

To address this, the updated system block diagram now includes a clear colour-coded legend defining the principal subsystem categories, including power sources, actuators, processing and control hardware, and communication modules. This makes the overall architecture easier to interpret and improves traceability between the system diagram and the later electrical and software sections. As a result, the revised figure presents the same hardware architecture as the PDR, but in a clearer and finalised form.

1.2.2 Feedback of Mechanical Design part

In the PDR, feedback on the mechanical design indicated that the CAD drawings lacked sufficient dimensioning. In response, this report presents updated three-view drawings of the robot, including critical dimensions such as overall length and width, as well as mounting locations for sensors and the robotic arm, to ensure manufacturability and assembly accuracy.

1.2.3 Software Architecture Diagram Refinement

Feedback from the PDR indicated that, although the software block diagram was clear, the **data flows and system interactions were not sufficiently explicit**, particularly regarding how the mission logic determines when navigation goals have been reached.

To address this, the software architecture diagram was refined to provide a clearer and more complete representation of inter-node communication. The following key improvements were implemented:

- **Explicit navigation feedback:** A dedicated `/navigation_status` feedback path was added from the Nav2 stack to the mission logic. This explicitly represents how the mission state machine determines whether a waypoint has been successfully reached or if navigation has failed, directly addressing the ambiguity identified in the feedback.
- **Clear separation of data flows:** The updated diagram distinguishes between different types of system interactions using structured topic flows:
  - sensor and perception data (e.g. LiDAR, RGB-D, odometry)
  - high-level mission decisions and navigation goals
  - low-level actuation commands for the base and manipulator
  - execution feedback from the manipulator
- **Improved mission logic representation:** The mission logic block was expanded to explicitly include:
  - the finite state machine structure
  - object–bin matching
  - navigation goal selection
  - grasp and drop management

This provides a clearer link between perception inputs and decision-making outputs.

- **Addition of object memory:** A dedicated **object memory module** was introduced within the mission logic, showing how detected object and bin positions are stored and updated. This reflects the implemented hashmap-based approach used for pairing objects with bins before initiating navigation.
- **Manipulator interaction clarity:** The command and feedback interfaces between the mission logic and the arm controller were explicitly separated into:
  - command topics (e.g. grasp pose)
  - execution feedback (e.g. grasp status)

  This improves clarity of the manipulation pipeline and its integration within the autonomy stack.
- **Addition of a diagram legend:** A colour-coded legend was introduced to clearly define the meaning of different arrow types (sensor data, decision flows, control commands, and feedback). This significantly improves readability and reduces ambiguity in interpreting system interactions.

These modifications ensure that the software architecture diagram accurately reflects the implemented system, clearly communicates data flow between components, and explicitly represents feedback mechanisms required for reliable state transitions within the mission logic.

1.2.4 Mission Planning Implementation through Software

The PDR feedback highlighted that, while the need for a mission planner was identified, its implementation and behaviour were not sufficiently described. This was primarily because, at the time of the PDR, the full mission logic had not yet been finalised.

In the final design, a complete **mission planner** has been implemented as a finite state machine and is explicitly represented in both the software architecture diagram and the corresponding section in the report. The planner operates on a **semantic memory structure (hashmap)** that stores detected object and bin locations, and initiates task execution only when a valid object–bin pair of matching colour is available.

The mission proceeds through a sequence of states: *exploration and mapping*, *object–bin pair selection*, *navigation to object*, *grasp execution*, *grasp verification and retry*, *navigation to bin*, and *object placement*. Transitions between these states are explicitly driven by system feedback, including

continuous perception updates, navigation status signals (goal reached/failed), and grasp success verification based on detection persistence.

This closed-loop control structure ensures coordinated interaction between perception, navigation, and manipulation, and allows the system to repeatedly execute the full cycle until all three objects have been successfully collected and deposited.

The mission planning logic is therefore now fully specified, integrated within the system architecture, and described in detail in *Mission Planning and Task Coordination*, directly addressing the limitations identified in the PDR feedback.

1.2.5 Project Plan

The Project Plan has **improved task allocation** within the team according to the feedback. In earlier versions, several activities scheduled for S2 were not clearly assigned to specific team members. To address this issue, responsibilities have now been defined in greater detail, ensuring that each task has a designated owner.

It should be noted that this allocation represents an initial plan for the upcoming stages of the project. As the project progresses and new technical challenges arise, the task distribution may be further adjusted to better align with development needs and team capacity.

1.2.6 Requirements Verification Matrix

The Requirements Verification Matrix has been revised to improve clarity and traceability. **Assumptions and constraints** have been separated from functional requirements to provide a clearer structure and to avoid ambiguity in requirement definitions.

In addition, **several requirements** have been refined and reorganized to make them more specific and measurable. This ensures that each requirement can be verified more effectively during system testing.

The **success criteria and corresponding verification cases** have also been updated. By defining more explicit acceptance criteria and test procedures, the matrix enables requirements to be tested and validated in a more straightforward, systematic manner.

1.3 Modifications made since the PDR

1.3.1 System Design

Since the PDR, the overall hardware architecture of the robot has remained unchanged, as the selected combination of the Leo Rover, myCobot 280 Pi manipulator, ASUS NUC 13 PRO, RPLIDAR A2M12, Intel RealSense D435, ethernet hub, and distributed power arrangement already provided a suitable basis for navigation, perception, manipulation, and communication.

The main modification in the final design is therefore not a change in subsystem selection, but a refinement in how the system is specified and presented. In particular, the external power supplies are now explicitly defined as dedicated Anker Prime power banks for the NUC and manipulator, and the system block diagram has been updated with a legend to make subsystem roles clearer. The system section has also been updated to align component functions and justifications with the finalised requirements.

These changes improve the clarity and completeness of the final system documentation while preserving the modular hardware design established at the preliminary design stage.

1.3.2 State Control Flow and System Software Integration

Following the Preliminary Design, the system software has been further refined and integrated, mainly focusing on **formalizing the control logic, establishing clear data processing procedures, and ensuring smooth interaction among subsystems**.

The main control module has finalized the state machine and the output control matrix. All key modules have reached a stable stage, with their interfaces and data structures defined. Data from sensors and actuators is now routed systematically, providing a solid foundation for implementation, testing, and further development.

1.3.3 Software Design: LiDAR-Based SLAM and Localisation

Since the PDR, the SLAM subsystem has been refined to improve robustness, data quality, and real-time performance. A key modification is the introduction of a **LiDAR filtering node**, which removes erroneous measurements caused by wheel reflections before publishing filtered scans on `/scan_filtered`. This ensures that both the SLAM module and navigation stack operate on cleaner input data, improving map consistency and localisation reliability.

The SLAM configuration has also been explicitly defined to operate in **online asynchronous mode**, prioritising recent sensor data to maintain real-time performance alongside computationally demanding components such as perception and navigation. In addition, the occupancy grid resolution has been fixed at **5 cm** $\times$ **5 cm**, balancing environmental detail with computational efficiency.

Further refinements include a clearer definition of the **transform tree** and a more explicit description of the **sensor fusion process**, distinguishing the role of LiDAR scans in providing spatial constraints for scan matching and wheel odometry in stabilising motion estimation between updates.

The computational design has also been clarified: while SLAM was already assigned to the NUC, its role has been explicitly justified in the context of concurrent execution with object detection and navigation, reinforcing the necessity of offloading SLAM from the onboard Raspberry Pi.

Additionally, the selection of a LiDAR-based SLAM approach over visual SLAM is now explicitly justified based on environmental characteristics and sensor allocation within the system.

These modifications and refinements improve the robustness, clarity, and justification of the SLAM subsystem while maintaining compliance with localisation and mapping requirements **(FR4, PR4.1–PR4.2)**.

1.3.4 Software Design: Navigation and Path Planning (Nav2)

Since the PDR, the navigation subsystem has been refined through both architectural additions and controller-level design changes. A key modification is the integration of **frontier-based exploration** using the **explore-lite** package, enabling autonomous exploration of unknown environments by generating navigation goals at frontier regions. This was not explicitly defined in the PDR and now provides a structured approach to environment coverage.

At the control level, the local planner has been changed from the **Dynamic Window Approach (DWB)** to the **Regulated Pure Pursuit (RPP)** controller. This decision was made following comparative testing, where RPP demonstrated more stable and smoother path-following behaviour for the differential-drive platform, while maintaining compliance with obstacle avoidance and velocity constraints.

The navigation strategy for target approach has also been revised. In the PDR design, safety margins were adjusted dynamically through costmap inflation and velocity scaling. In the final system, costmap parameters are kept fixed, and a **standoff offset** is instead incorporated directly into the goal generation process. This allows the robot to stop within the required **30–35 cm range (PR7.1)**

while ensuring consistent and predictable behaviour, improving alignment with the manipulator workspace **(FR6)**.

Additionally, the separation between **exploration and task-driven navigation** has been made explicit, with exploration handled by the frontier-based module and Nav2 responsible for executing navigation goals generated by the mission planner.

These modifications improve the robustness, clarity, and predictability of the navigation system while maintaining compliance with requirements **(FR4, FR7–FR8, FR17, PR7.1, PR8.1–PR8.2)**.

1.3.5 Software Design: RGB-D Vision: Object and Bin Detection

Since the PDR, no fundamental design changes were made to the RGB-D perception subsystem. The existing approach, based on YOLOv8s and RGB-D fusion, was retained and demonstrated to perform reliably under the expected operating conditions. The section has been refined to provide a clearer description of the processing pipeline, including depth-based 3D localisation and data usage across subsystems. The class labelling strategy and the use of a single 3D centre point (instead of a full pose) are now more explicitly justified based on task requirements and computational efficiency. These refinements improve clarity and system integration while maintaining compliance with the specified requirements.

1.3.6 Manipulation Coordination and Control

Since the PDR, the manipulation subsystem has undergone a significant architectural redesign, shifting from a **centralised planning approach** on the NUC to a **distributed control architecture**.

In the PDR design, the NUC performed inverse kinematics (IK) and generated grasp poses. In the final system, the NUC performs only **high-level coordination**, sending the detected object position (in the camera frame) together with simple manipulation commands (e.g. gripper open/close, move to pose, move to initial pose).

All low-level computation is executed on the **arm's onboard Raspberry Pi**. The arm controller first transforms the received object position from the `camera_link` frame into the **arm base frame**, and then uses the **pymycobot** library to compute inverse kinematics and execute the grasp. These computations are lightweight and can be performed reliably in real time on the arm controller.

This redesign reduces computational load on the NUC, improves system responsiveness, and avoids unnecessary communication overhead for time-critical control loops. It also simplifies the overall

pipeline by removing the need for explicit IK planning and trajectory generation on the NUC.

Additionally, the manipulation strategy has been simplified. A fixed end-effector orientation is used for grasping, eliminating the need for complex motion planning and collision checking. The use of the lightweight **pymycobot** interface was selected over frameworks such as MoveIt2, which were evaluated but rejected due to unnecessary computational overhead for the given task.

These changes result in a more robust, modular, and efficient manipulation pipeline while maintaining compliance with manipulation and safety requirements **(FR11–FR14, FR17, FR20–FR21)**.

1.3.7 Modification of The Mechanical Part

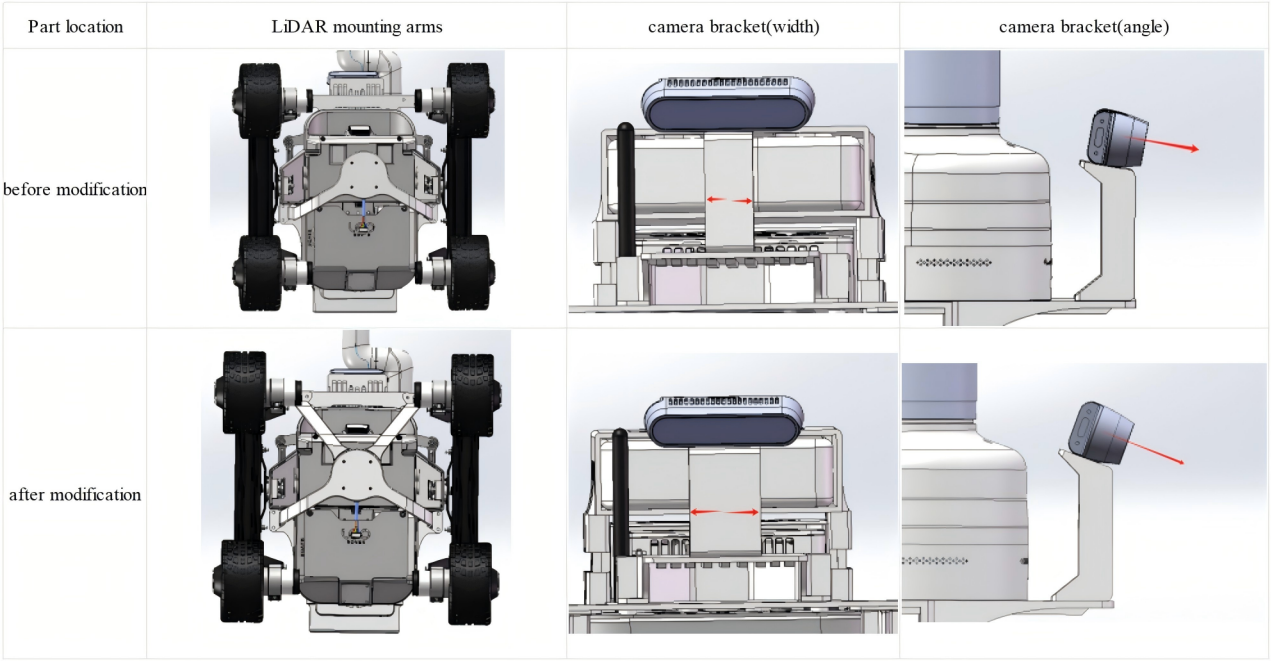


Fig. 1. Modification of Sensor bracket

- **Sensor range and stability:** Regarding sensor integration, testing revealed that the originally designed structure exhibited significant deformation and vibration during actual robot operation—for example, excessive tilt of the LiDAR mounting angle led to inaccurate localization, while camera vibration compromised recognition accuracy. In response, the following modifications were implemented:
  - the number of LiDAR mounting arms was increased from two to four to enhance structural stiffness.
  - the width of the camera bracket was broadened and reinforcement ribs were added to improve rigidity.

With respect to sensor coverage, the initial camera configuration was constrained by its field of view, limiting its ability to detect objects in close proximity to the robot and imposing excessive demands on navigation precision. the following modifications were implemented:

- the camera position was slightly shifted rearwards and its mounting angle was increased by 10°.

Consequently, reducing the minimum detectable distance and improving the reliability and adaptability of the perception system.

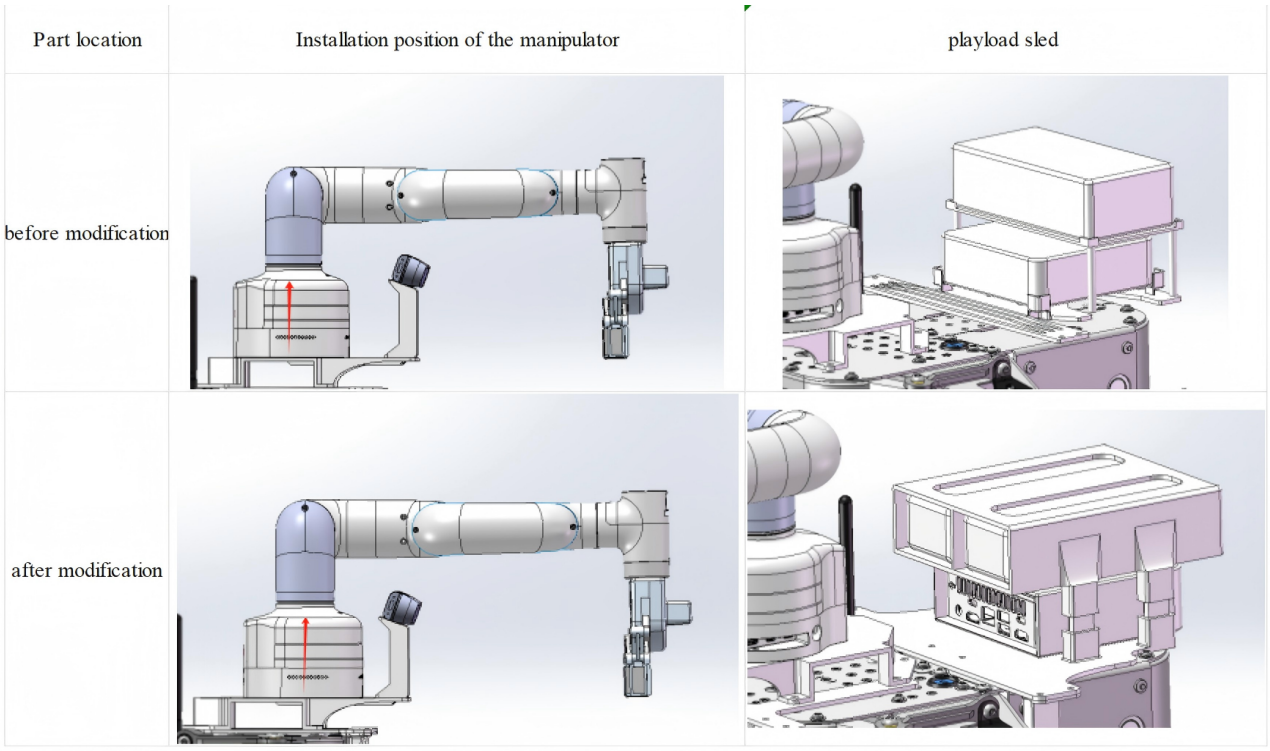


Fig. 2. Modification of payload sled

- **The manipulator workspace:** The manipulator workspace has been optimized to ensure a safe distance between the robot and the target box. This change enables the grasp of objects at a greater distance, thereby enhancing operational safety and flexibility.
  - The mounting position of the manipulator was shifted 20 cm forward.
- **payload sled design:** The mounting configuration for the battery and NUC was revised within the updated payload sled design to accommodate the new battery dimensions and quantity. A T-slot mounting mechanism was adopted in place of bolted connections, allowing quick and convenient installation and removal.
- **Structural strength and rigidity:** With regard to structural strength and stiffness, as certain components have become increasingly complex, verification based on simplified models may introduce considerable errors. To address this, finite element analysis (FEA) was adopted to

compute the deformation and maximum stress under realistic operating conditions, enabling a more accurate assessment and validation of structural performance.

1.3.8 Dual-Battery Power System Updated

The original two TGR-PP-24V-5000-B 24V battery packs and step-down modules will be decommissioned and replaced by a distributed dual-battery architecture. We have introduced identical Anker Prime (26K, 300W) power banks to serve as independent power sources for the computing unit (ASUS NUC 13 Pro) and the execution unit (myCobot 280 Pi robotic arm), respectively. By utilizing the type-c to DC cable with the digital negotiation mechanism of the USB PD protocol to provide customized voltage directly to the devices, this architecture significantly reduces physical volume and wiring complexity.

For the ASUS NUC 13 Pro compute node, the connection scheme utilizes a high-specification Type-C to DC 5.5*2.5mm PD trigger cable with a built-in E-Marker chip. The hardware control chip of this cable is factory-set to a 20V trigger logic and supports a maximum current of 5A. This ensures the link can continuously and stably deliver up to 100W of input power to the NUC. For the myCobot 280 Pi robotic arm□this link is equipped with a Type-C to DC 12V fixed-voltage PD trigger cable. Constrained by the standard USB PD protocol specifications, the maximum output current for the 12V tier is typically limited to 3A.

2 Sustainability Checklist

2.1 Materials

The system is built from a combination of **commercial robotic subsystems** and **custom mounting structures**. The main hardware consists of the Leo Rover mobile base, ASUS NUC 13 Pro, myCobot 280 Pi manipulator and gripper, RPLIDAR A2M12, Intel RealSense D435, Ethernet hub, and external power banks. The main material groups are **aluminium structural elements**, **PLA housings**, cable ties, and electronic materials within the PCBs, sensors, and compute hardware.

Most of the structural mass is in the rover chassis and manipulator. Aluminium is advantageous because it provides **low mass, good stiffness and recyclability**. For a mobile robot, lower mass is important because it reduces the energy required for locomotion. The estimated complete system mass is approximately **10.788 kg**, indicating that weight has already been constrained within the design limits.

Custom parts such as brackets and supports are **3D printed**, primarily in **PLA Basic**, with simplified geometry and reduced unnecessary features. This reduces print time, material use, and manufacturing waste during iteration. PLA is preferable to some conventional plastics because it is **bio-derived**, although it still has limitations in durability and end-of-life handling.

The main limitation is that most major subsystems are **off-the-shelf**, so their embodied carbon, sourcing, and internal material composition are outside the scope of the project. Components such as the NUC, camera, LiDAR, and manipulator contain **mixed electronic materials** that are difficult to separate and recycle.

The design does support **partial disassembly**. Components are mounted separately using **standard screws and cabling** rather than permanent bonding, so the LiDAR, RealSense, NUC, and manipulator can be removed independently for **repair, replacement, or reuse**.

Overall, the design uses durable materials and reduces unnecessary material use in custom parts, but sustainability is limited by reliance on pre-manufactured electronic subsystems. The main improvements would be to minimise printed material further, reuse mounts where possible, and use recycled feedstock where available.

2.2 Software

The software architecture is implemented as a **distributed ROS2 Jazzy system**. The Intel NUC performs the **computationally intensive tasks**, while the Leo Rover Pi and myCobot Pi handle low-level actuation locally. The NUC runs SLAM, **YOLO-based perception**, Nav2 and path planning, while the rover Pi manages base motion and odometry and the manipulator Pi executes inverse kinematics and gripper commands through the use of its pymycobot library.

Software sustainability is mainly determined by **processing load, communication overhead, and storage**. The current design reduces these overheads by processing image and LiDAR data **locally on the NUC** and publishing only **task-relevant outputs**. This avoids streaming raw sensor data across the system and reduces duplicated computation between nodes.

The selected software also reflects **computational efficiency**. YOLOv8s was chosen because it can **run in real time on the NUC CPU**, and SLAM Toolbox was selected partly because it provides suitable indoor mapping performance at **relatively low CPU cost**. These choices reduce the need for more power-intensive hardware.

The main trade-off is that the system maintains **continuous perception, localisation, and navigation updates** throughout operation. Image processing, YOLO inference, SLAM, TF updates, and

Nav2 together create a **constant baseline computational load**. This is not minimal in terms of energy use, but it is necessary for reliable autonomy in an unknown environment.

Further improvement would come from **adaptive computation** rather than architectural change. Camera processing could be reduced when no candidate objects are present, update rates could be adjusted based on motion state, and logging could remain limited to task-relevant outputs unless debugging is required.

Overall, the software is reasonably efficient for the task because it **keeps processing onboard**, avoids cloud dependence, and allocates tasks to suitable compute units. The main remaining sustainability issue is the continuous processing load.

2.3 Energy

Energy efficiency is a **major sustainability issue** because the system combines locomotion, continuous sensing, onboard computation, and manipulation. The Leo Rover uses its onboard battery, while **two additional power banks** are used: one for the ASUS NUC and one for the myCobot manipulator. This separation improves **power stability** and prevents compute performance being affected by manipulator loads.

The power budget shows that demand is distributed across **three subsystems**: the rover base, the NUC with the RealSense and LiDAR, and the manipulator. The estimated **total peak load is 268.88 W** and a **typical operating load of approximately 156.5 W**. This shows that operational energy demand is driven by the combined effect of mobility, sensing, computation, and actuation rather than by a single component.

Some energy-conscious decisions are already present. Heavy processing is **centralised on the NUC** rather than duplicated across smaller boards, and **separate power supplies** improve reliability and reduce the chance of failed runs or hardware stress. The **20 minute mission limit** also bounds total energy consumption per task cycle.

However, the system is not explicitly **energy optimised**. Sensors and onboard compute remain active throughout the mission, and there is no **low-power mode, adaptive sensing, or selective shutdown of idle subsystems**. This means energy is consumed continuously even when only part of the system is actively needed. For example, the manipulator and its Raspberry Pi may remain powered during exploration, and the camera and LiDAR may continue operating at full rate during stationary phases.

Potential improvements include **lower-power idle states**, adaptive sensing rates, and reduced subsystem activity when hardware is not in use. More efficient route planning would also reduce rover motion and therefore base energy consumption.

Overall, the energy system is robust and well justified, but it prioritises reliability over minimisation of energy use. The main sustainability improvement would be **smarter subsystem power management**.

2.4 Waste

The robot produces almost no **direct material waste** during mission execution because it does not consume disposable materials or generate physical by-products while operating. The main waste issue is therefore **lifecycle waste** from prototyping, maintenance, failure, and end-of-life disposal rather than operational waste.

Custom parts were manufactured using **3D printing**, which is efficient for **small-batch iteration** and avoids the material waste associated with more subtractive manufacturing methods. However, failed prints, revised designs, and discarded prototypes still generate waste, particularly during integration of brackets, mounts, and support structures.

Electronic waste is the more significant long-term issue. The system relies on the NUC, RealSense camera, RPLIDAR, Raspberry Pis, Ethernet hub, and battery packs, all of which contain **mixed materials** and are difficult to recycle at component level. Batteries are particularly important because they **degrade over time** and require controlled disposal.

The **modular design** reduces waste by allowing **individual parts to be replaced** rather than discarding the full system. A damaged mount can be reprinted, a failed sensor can be replaced independently, and the manipulator or NUC can be removed without redesigning the whole platform. This improves repairability and extends overall system life.

The limitation is that no formal strategy is described for **reuse of failed prints**, segregation of electronic waste, or **battery end-of-life handling**. In practice, failed electronics are likely to be replaced rather than repaired, which is convenient but less sustainable.

Overall, operational waste is low, but lifecycle waste remains due to prototyping and electronics. The main improvements would be reducing unnecessary reprints, reusing printed parts where possible, and ensuring **proper recycling of batteries and failed electronics**.

2.5 Emissions

The robot produces no **direct combustion or exhaust emissions** during operation because it is fully electrically powered. This is particularly suitable for **indoor deployment**, where air quality and user safety are important.

The main environmental issue is therefore **indirect emissions**. These arise from electricity used to charge the batteries and power banks, **embodied emissions** from manufacturing the rover, manipulator, sensors, and compute hardware, and **transport emissions** associated with procuring the components. Of these, manufacturing is likely the **largest fixed contributor** because the NUC, RealSense, LiDAR, and battery packs are relatively complex electronic devices with energy-intensive production processes.

Operational emissions are **directly linked to energy use**. With a typical running power of approximately **156.5 W**, longer mission duration or inefficient processing increases indirect emissions through electricity consumption. Although the mission itself is short, repeated testing and debugging over the full project lifecycle can still result in a **significant cumulative energy demand**.

Some design choices reduce emissions indirectly. The system is compact, with a total mass of around **10.288 kg**, which lowers transport burden and reduces locomotion energy compared with a larger platform. **Onboard processing** also avoids dependence on cloud computation or additional external infrastructure. Reuse of the same platform for multiple tasks would further spread the embodied emissions of manufacture over a longer service life.

However, no **explicit emissions-reduction strategy** is included. There is no renewable charging provision, no carbon-aware operating policy, and no attempt to minimise embodied emissions through **bespoke component selection**.

Overall, the robot achieves **zero direct operational emissions**, but still has indirect emissions from electricity use, manufacturing, and potentially transport across long distances. The most practical improvements would be reducing idle energy demand, extending hardware lifespan, and reusing the platform beyond the project.

2.6 Communications

The system minimises unnecessary data transmission by keeping all **high-bandwidth processing local to the Intel NUC**. RGB-D images and LiDAR data are processed onboard, and only **compact outputs** are shared between subsystems. This avoids transmitting raw sensor streams, which are

significantly larger than the final outputs and would increase both energy consumption and system complexity.

The perception pipeline operates at approximately **10 Hz**, but instead of publishing images or dense detections, it outputs only a **detection flag and a 3D object position** (a small set of numerical values). This reduces inter-node communication to the minimum required for decision-making. The same output is reused for both navigation and manipulation, ensuring the information is **computed once and not duplicated** across the system.

Communication complexity is further reduced by **limiting the number of ROS2 topics** and reusing interfaces across different stages of the mission. The arm subsystem exposes only **three commands** (gripper control, signal to move to initial pose, or target location), which are reused for both grasping and object release. Similarly, perception outputs are reused for grasp verification, avoiding redundant message passing and reducing processing overhead.

The navigation stack requires continuous data flow from LiDAR and SLAM. Rather than reducing the update rate, the system **filters LiDAR data before it is used by costmaps**, removing invalid readings and reducing unnecessary downstream computation. The update rate ($\sim 10\,$Hz) was selected as a balance between responsiveness and computational load.

An **event-driven approach** is used for high-level control, where commands such as mission start are transmitted only when required. This avoids continuous communication and reduces idle processing.

Processing every camera frame increases computational load but improves detection reliability and reduces the likelihood of failed grasps or repeated navigation. This reduces the overall energy cost of completing a mission, despite higher per-step computation.

The system does not store **raw sensor data** during operation, significantly reducing storage requirements. Continuous storage of RGB-D images and LiDAR scans would quickly lead to large data volumes, especially at 10 Hz, whereas the current approach only retains compact task-relevant outputs. However, this limits post-mission analysis and debugging. Additionally, no dynamic throttling or adaptive communication strategies are implemented, so some redundancy may still exist.

2.7 Modularity

The system is designed with a **modular architecture at both hardware and software levels**, allowing individual components to be replaced, upgraded, or repaired without affecting the entire robot.

At the hardware level, the robot is composed of **clearly separable subsystems**, including the Leo Rover base, Intel NUC, RGB-D camera, LiDAR, and myCobot manipulator. These components are connected through **standard interfaces such as USB and Ethernet**, enabling faulty parts to be replaced without redesigning the system. For example, the RGB-D camera or LiDAR can be swapped independently, and the manipulator can be removed or replaced without modifying the base platform.

Modularity is further reinforced through the use of **independent compute units**. High-level processing is executed on the NUC, while low-level control of the rover and manipulator is handled by their respective Raspberry Pis. This separation improves **fault isolation**, as failures in one subsystem do not propagate to others, and simplifies repair and replacement.

At the software level, the system is implemented as a **distributed ROS2 architecture**, where functionality is divided into **loosely coupled nodes**. Perception, navigation, mission logic, and manipulation are implemented as separate components that communicate through well-defined topics. This allows individual modules to be replaced or extended with minimal impact on the rest of the system, for example replacing the perception model or navigation controller.

The system also uses **abstracted control interfaces**. The manipulator is controlled through a small set of **high-level commands** rather than low-level joint control, allowing different hardware implementations to be integrated without modifying the overall system logic.

Some limitations remain. Physical integration of components introduces **constraints on accessibility**, and replacing sensors such as the camera may require recalibration of the perception pipeline. Additionally, while ROS2 provides loose coupling, some dependencies remain through shared message definitions and coordinate frames.

Overall, the system achieves a **high degree of modularity**, enabling efficient repair, component replacement, and system adaptation without requiring full system redesign.

2.8 Location/Placement

The robot is designed for **indoor deployment in controlled environments** and does not require fixed infrastructure or permanent installation. All components are integrated into a **compact and portable platform**, allowing the system to be easily deployed and redeployed without specialised equipment.

Transportation requirements are minimal, as the robot can be **manually carried for short distances** or moved within the same building using a trolley, wheeled platform, or lift. The total system weight

is **below 18 kg**, allowing it to be safely handled by a single person. Due to its relatively small size and modular design, no dedicated transport systems are required, reducing the environmental impact associated with deployment.

For safer handling and transport, sensitive components such as the manipulator and RGB-D camera can be detached and stored separately in **protective containers**, and then reattached to the rover when deployed. This reduces the risk of damage and extends the lifespan of key components. The LiDAR can remain mounted if the robot is transported in a protective enclosure, although additional packaging may be used if required.

From a deployment perspective, the system operates **entirely onboard** and does not rely on external infrastructure such as remote servers or cloud processing. This simplifies setup and reduces additional energy and communication requirements during operation.

The system is intended for **repeated use within the same environment**, avoiding frequent relocation and further reducing transportation-related energy costs. This makes the deployment process lightweight and sustainable compared to larger robotic systems that require significant logistical support.

For storage, the robot does not produce emissions or require special environmental conditions beyond standard indoor storage. However, **batteries should be stored separately** from the robot when not in use to improve safety and reduce the risk of damage or degradation. Sensitive components such as the manipulator, RGB-D camera, Intel NUC, and LiDAR should ideally be stored in **separate protective containers** to prevent mechanical damage and extend their lifespan. Protective casing is also recommended for the remaining system.

The most sustainable method of transportation is **manual transport**, as the robot can be carried without the need for a vehicle. If longer distances are required, transportation using **existing shared vehicles** is preferred over dedicated transport to minimise additional energy consumption. When transported externally, protective casing is required to prevent damage to exposed electronics.

2.9 Maintenance

The system does not follow a **formal scheduled maintenance procedure**, but instead relies on a combination of **pre-mission software checks** and regular manual inspection.

Before each mission, the system performs a **software-level validation** by ensuring that all required ROS2 nodes are active and communicating correctly. Each subsystem (perception, navigation, and manipulation) must publish the expected topics for the system to proceed. This implicitly verifies

that the underlying hardware components, such as sensors and actuators, are functioning, as node failure or missing data streams would prevent execution. These checks **reduce the likelihood of faults occurring during mission execution**.

In addition to software checks, the physical condition of the robot is monitored through **manual inspection**. Components such as cables, mounts, and connectors are visually checked to ensure they are secure and undamaged before operation, reducing the risk of **mechanical or connection failures during runtime**.

The system does not currently include a **dedicated diagnostic or health-monitoring module** (e.g., temperature monitoring, hardware status reporting, or predictive maintenance). As a result, failures are primarily detected at runtime rather than anticipated in advance, which introduces a **risk of mid-mission failure**.

During development, a complete failure of the Intel NUC was observed despite proper storage conditions, highlighting the limitations of relying solely on manual inspection and basic checks. This demonstrates that some failure modes cannot be detected without more advanced monitoring.

For future improvements, a more structured maintenance strategy could be implemented, including **automated diagnostics**, periodic hardware health checks, and logging of system performance over time to detect degradation. This would enable earlier fault detection and reduce the likelihood of unexpected failures during operation.

2.10 Repurposing

The robot can be repurposed both as a **complete system** and through reuse of its **individual components**, with only minor modifications required.

At the hardware level, each component can be reused in other robotic systems. The Intel NUC can be redeployed for **compute-intensive tasks** such as SLAM, perception, or multi-robot coordination. The RGB-D camera and LiDAR can be reused for perception and mapping in other mobile robots or fixed monitoring setups. The manipulator can be integrated into different platforms for pick-and-place or interaction tasks, while the rover base itself can be reused as a mobile platform for navigation, delivery, or inspection tasks in **flat indoor environments**.

The full robot system can also be repurposed for different applications with minimal changes. For example, it can be adapted for indoor delivery of **lightweight items** such as documents or tools in office or laboratory environments. By adding a storage container, the robot can transport **multiple items in a single mission**, enabling basic logistics or workspace organisation tasks.

The manipulation capabilities allow reuse in object handling tasks. The robot can be used for sorting or organising objects by detecting items and placing them in predefined locations such as shelves or containers. The current gripper can also be replaced with **alternative end-effectors**, such as softer or sensor-based grippers, to enable handling of more delicate objects.

The system can also be repurposed for inspection and monitoring. Using its existing navigation and perception pipeline, the robot can patrol indoor environments, follow predefined routes, and detect objects or changes in the environment. This requires only modification of the task logic, while the core system remains unchanged.

Additional functionality can be introduced through small hardware extensions. For example, adding a storage module enables **multi-object transport**, while integrating tactile or force sensors would improve manipulation reliability. These extensions build on the existing platform without requiring replacement of the core system.

At the software level, the navigation stack (SLAM and Nav2) and ROS2-based architecture can be reused directly across applications. The perception module can be retrained for different object categories, or replaced depending on task requirements. While software reuse improves development efficiency, the **primary sustainability benefit comes from reusing physical components**.

Overall, the robot supports repurposing at both system and component levels. This reduces **electronic waste** by enabling reuse of existing hardware and extends the functional lifespan of the platform.

3 Cyber Security Considerations

In this robotic system, the Leo Rover, collaborative robotic arm, vision camera, and power components are interconnected through wired Ethernet, with an Intel NUC acting as the central computing hub. All subsystems communicate within an isolated local subnet, enabling reliable low-latency data exchange while reducing exposure to external network threats.

3.1 Hardware Security and Network Isolation

Current Status: Communication between all primary subsystems relies on wired Ethernet connections operating within an isolated local subnet.

Challenges: Although wired communication reduces exposure to remote wireless attacks, unauthorized physical access to network interfaces or internal hardware may still pose security risks.

Mitigation: Network-level protection is enforced using Uncomplicated Firewall (UFW) rules with a default-deny policy and IP whitelisting. These measures restrict unauthorized connections, reduce the risk of port scanning, and mitigate potential Denial-of-Service (DoS) attacks.

Future Work: Further network segmentation, such as the introduction of VLAN-based separation, could improve monitoring capabilities and reduce the impact of potential internal anomalies.

3.2 System Integration and Network Interaction

Current Status: The robotic platform operates in a controlled environment based on Ubuntu 24.04, benefiting from established operating system security mechanisms.

Challenges: Since all components currently share a common trust boundary, a compromised node could potentially propagate malicious commands to other parts of the system.

Mitigation: Controlled access policies and logical segmentation within the network environment are applied to reduce the likelihood of unintended interactions between system components.

Future Work: Future work may explore adopting zero-trust security principles and implementing namespace-level isolation within ROS domains to further restrict unauthorized lateral communication.

3.3 ROS2 Middleware Communication

Current Status: Inter-process communication is implemented using ROS 2 with a DDS-based publish–subscribe architecture. Logical separation is achieved through a dedicated `ROS_DOMAIN_ID`.

Challenges: Standard DDS domain isolation does not provide encryption or node authentication, which may expose the system to packet sniffing, message spoofing, or rogue node injection within the network.

Mitigation: System security is currently maintained through strict network isolation and controlled topic usage within the ROS domain.

Future Work: The deployment of SROS2 is planned to enable node authentication, encrypted message exchange, and fine-grained access control for improved communication security.

3.4 Data Privacy and Management

Current Status: The vision processing pipeline operates entirely at the edge, processing image data directly in RAM without persistent local storage or external transmission.

Challenges: Despite local processing, improper handling of temporary data buffers or system logs could potentially expose sensitive visual information.

Mitigation: Data minimization principles are applied, ensuring that only essential outputs such as detected object coordinates are retained and shared with other system components.

Future Work: Future improvements may include secure logging mechanisms and improved monitoring of data handling processes to further strengthen privacy protection.

3.5 Software Version Control and Access Management

Current Status: All software and configuration files are maintained on GitHub using structured version control and role-based repository access.

Challenges: Unauthorized repository access or accidental exposure of sensitive configuration data could introduce security vulnerabilities.

Mitigation: Repository access is strictly controlled and supported by code review processes to prevent the inclusion of sensitive credentials or configuration details.

Future Work: Integrating automated DevSecOps tools, such as dependency vulnerability scanning and security checks within continuous integration pipelines, may further improve software security.

3.6 Cyber-Physical Fail-safe Mechanisms

Current Status: The robot includes a hardware-level Emergency Stop (E-Stop) circuit that operates independently from the operating system and network layers.

Challenges: If the system becomes compromised or communication is disrupted, incorrect control commands or loss of control signals could result in unsafe robot motion.

Mitigation: The E-Stop allows operators to immediately disable all actuators. In addition, the system monitors control signals to detect potential communication loss during operation.

Future Work: Additional hardware safety mechanisms and enhanced monitoring strategies may be explored to further improve system reliability and operational safety.

4 System

This section describes the overall hardware of the robot, from the mobile base to the manipulator, sensors, processing, and communication. This system is built with the Leo Rover as the mobile platform, which has a myCobot 280 Pi manipulator, an ASUS NUC 13 PRO, RPLIDAR A2M12, and an Intel RealSense D435 depth camera mounted on it. The Rover has its own battery, while two Anker Prime power banks are used to power the manipulator and NUC. An Ethernet hub connects each component of the system and the Rovers on board Wi-Fi provides a wireless connection.

4.1 System Block Diagram

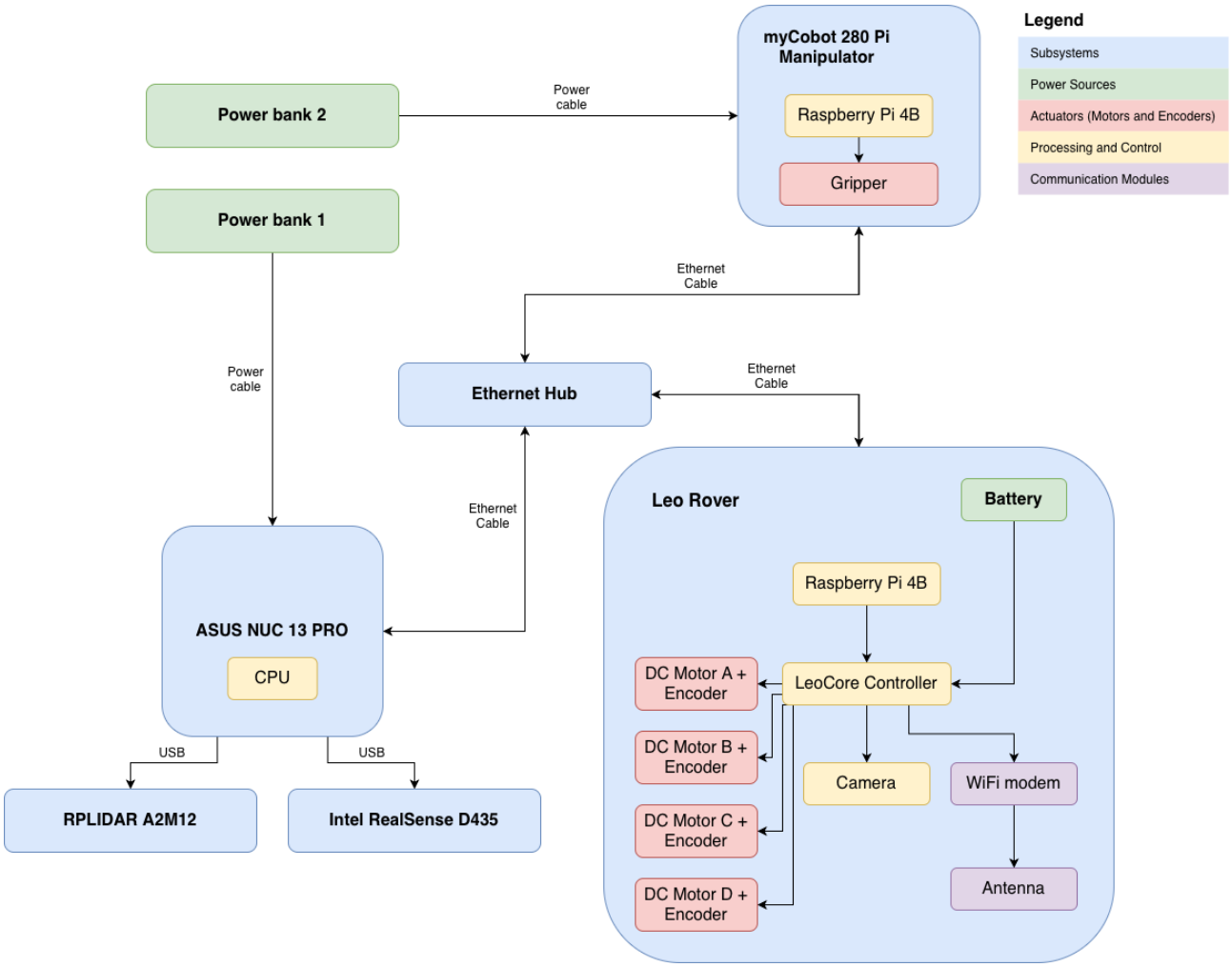


Fig. 3. System block diagram of the overall robot hardware architecture, showing the Leo Rover mobile base, ASUS NUC 13 PRO, myCobot 280 Pi manipulator and gripper, RPLIDAR A2M12, Intel RealSense D435, ethernet hub, communication links, dual Anker Prime power banks, and the colour coded categories.

4.2 Components

4.2.1 Leo Rover 1.8 mobile base

The Leo Rover 1.8 provides the mobile platform and is powered by its own battery. It integrates four DC motors with encoders to control the wheels alongside a Raspberry Pi 4B, onboard camera and Wi-Fi modem. It offers a sufficient payload of 5 kg to carry the manipulator, NUC, sensors and power banks while being able to comfortably operate in its workspace.

Requirements analysis

- **FR1 / PR1.1** - provides the mobile platform for autonomous mission execution which the NUC can control without human intervention.
- **FR3** - supports a modular hardware design by acting as the main base onto which key components can be mounted and replaced.
- **FR4, FR5** – wheel encoders and LeoCore provide odometry for localisation to ensure a path without collisions can be taken.
- **FR17 / PR17.1, PR17.2** – the rover can be configured to remain within the required linear and angular speed limits.
- **FR18 / PR18.1** – total system mass remains within the 18kg limit.

4.2.2 ASUS NUC 13 PRO

The ASUS NUC 13 PRO is the central onboard computer which handles most of the processing. It has significantly more processing power compared to Raspberry Pi boards embedded in the rover and manipulator. This is important for processing LiDAR scans and RGB-D data simultaneously in real-time while operating in the environment. It also offers multiple USB 3.0 ports and an ethernet port allowing for a complete connection to every component. The NUC is powered by its own dedicated Anker Prime power bank to ensure power spikes due to motor activity and other components do not affect it.

Requirements analysis

- **FR1 / PR1.1, PR1.2** - the NUC runs the autonomous software stack required to complete the mission without human intervention and within the time limit.

- **FR4 to FR10** – mapping, localisation, object detection and navigation processing algorithms will run on the NUC
- **FR16 / PR16.1, PR16.2** – the NUC supports reliable and low-latency communication across the robotic system.

4.2.3 myCobot 280 Pi manipulator and gripper

The myCobot 280 Pi is a 6 DoF robotic arm mounted on the Leo Rover. It has its own integrated Raspberry Pi 4B and is powered by a second dedicated Anker Prime power bank. The gripper attached provides the end effector needed to grasp objects. Its payload and reach are enough to meet the requirements and hold the object for the intended duration of time.

Requirements analysis

- **FR6** - the manipulator workspace defines the target region that the mobile base must reach.
- **FR11 / PR11.1** - the manipulator and gripper provide the grasping capability required for objects located within reachable workspace.
- **FR12 / PR12.1** - the manipulator supports repeated grasp attempts when commanded by the mission logic.
- **FR13** – the gripper securely holds target objects during stationary and mobile operation.
- **FR14 / PR14.1** – the manipulator enables deposition of objects into the assigned colour-matched bin.

4.2.4 RPLIDAR A2M12

The RPLIDAR A2M12 measures distance across the horizontal plane by emission of light beams and time of flight calculations. LiDAR scanning has low processing costs while providing accurate distance measurements than a vision camera alone, enabling collision avoidance. The RPLIDAR connects to the NUC via USB, where it is powered and data is integrated into the ROS stack for obstacle avoidance and localisation.

Requirements analysis

- **FR4 / PR4.1, PR4.2** - LiDAR data supports mapping and localisation accuracy.
- **FR5 / PR5.1 to PR5.3** – LiDAR enables active obstacle detection within the required range and clearance limits.
- **FR6** – LiDAR supported navigation helps position the robot within the manipulator workspace.

4.2.5 Intel RealSense D435

The Intel RealSense D435 is an RGB-D camera which provides colour and depth images of its field of view. It enables object and bin detection, as well as depth-based localisation of targets relative to the robot. It is connected directly to the NUC via USB where the data can be passed to the ROS stack and processed accordingly.

Requirements analysis

- **FR7 / PR7.2** – depth and colour images enables detection and localisation of objects for approaching.
- **FR8** – depth and colour images support localisation and autonomous approach to the target bin.
- **FR9 / PR9.1, PR9.2** – colour and depth information support detection of the three target object colour classes.
- **FR10 / PR10.1, PR10.2** – colour and depth information support detection of the three target bin colours.

4.2.6 Anker Prime Power banks

Two external Anker Prime power banks are used to keep the power separate for processing and manipulation, reducing the chances of inconsistent voltage or noise in one subsystem affecting another. The use of power banks makes system integration, charging and replacement easy.

Requirements analysis

- **FR15 / PR15.1** – the external power banks contribute to continuous and regulated power for the full mission duration plus reserve.
- **FR19 / PR19.1** – the power subsystem is integrated within an electrical design that includes shielding and isolation of exposed electrical connections.
- **FR20 / PR20.1** – the robotic system includes a physical power off switch through the Leo Rover battery system, while the NUC, manipulator, and sensors can be disconnected by their external power cables in an emergency.

4.2.7 Communication links

Internal communication between the NUC, Leo Rover and myCobot manipulator is handled over ethernet across the local network. This ensures low latency and high bandwidth for control commands, odometry and data streams in real-time. The Leo Rovers Wi-Fi provides an external communication bridge which the operator can connect via and run commands without a physical connection. The ethernet hub is a central point which connects the NUC, myCobot and Leo Rover to ensure they're available on the same local network. This allows for seamless and reliable communication between components and sharing of ROS topics.

Requirements analysis

- **FR1 / PR1.1** – the Wi-Fi link allows for a single start command at the start of the autonomous mission.
- **FR16 / PR16.1, PR16.2** – ethernet and Wi-Fi support reliable, low-latency communication between subsystems.
- **FR21 / PR21.1** – Wi-Fi and ethernet will communicate the stop command to ensure the system shuts down within the required response time.

5 Mechanical Design

5.1 Overall design

The CAD model illustrates the mechanical design of the robot. The following factors represent the primary design challenges and constraints:

- **Environmental Constraints:** The height of the bin for object placement is only 210 mm. This necessitates mounting the LiDAR at a height below 210 mm to enable detection of the bin for navigation and obstacle avoidance. Simultaneously, the manipulator should be installed as close as possible to this height to maximize its operational workspace for grasping.
- **Weight Distribution:** The overall center of gravity (CoG) of the robot should be positioned close to its geometric center to prevent tipping or instability. This requires a balanced distribution of major components such as the manipulator, NUC, and battery.
- **Grasping Task:** To achieve a larger workspace for grasping, the manipulator is positioned towards one side of the rover rather than centrally. Furthermore, the placement of other sensors must avoid interference with the manipulator's operational envelope.
- **Spatial Allocation:** The available mounting space on the Leo Rover platform is limited. Both the NUC and the manipulator require numerous cable connections (power, sensors, I/O devices), necessitating dedicated space for cable routing and connector management.
- **Sensor Coverage and Stability:** The placement must ensure that the target object remains within the camera's field of view (FOV) at appropriate distances, and that the LiDAR maintains sufficient coverage for environmental perception.
- **Manufacturing and installation:** All components must be capable of being obtained through 3d printing or sheet cutting. Each part should not be overly complex, and the installation should be simple and easy to replace.

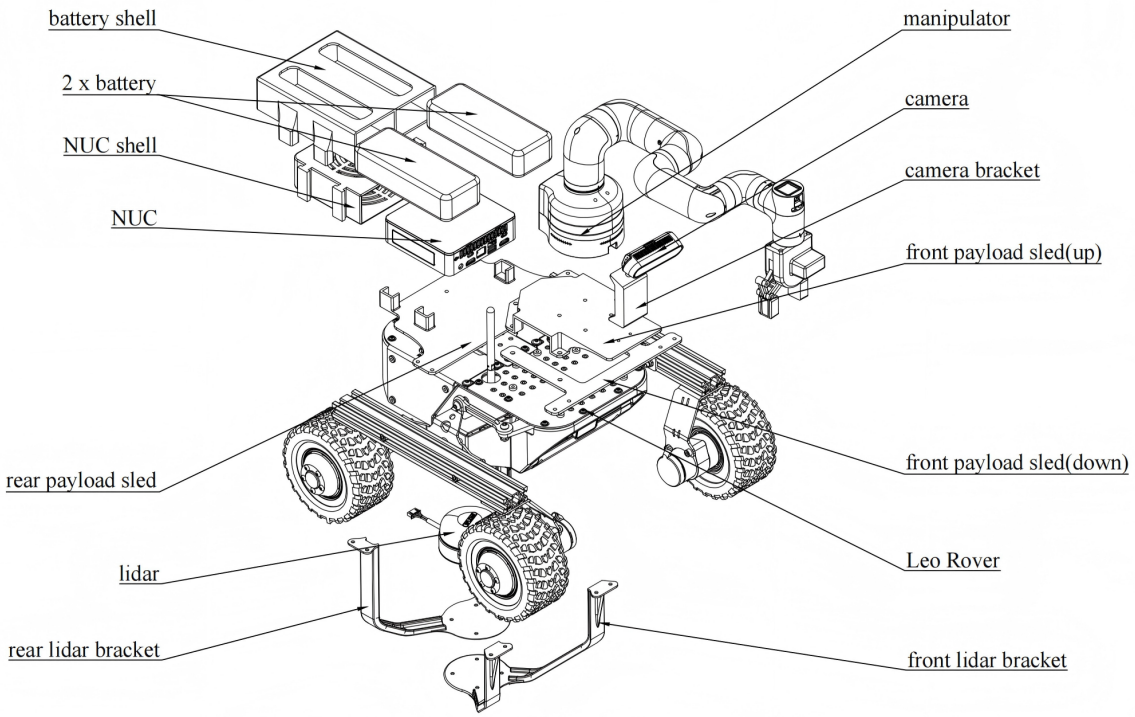


Fig. 4. Exploded Assembly Drawing

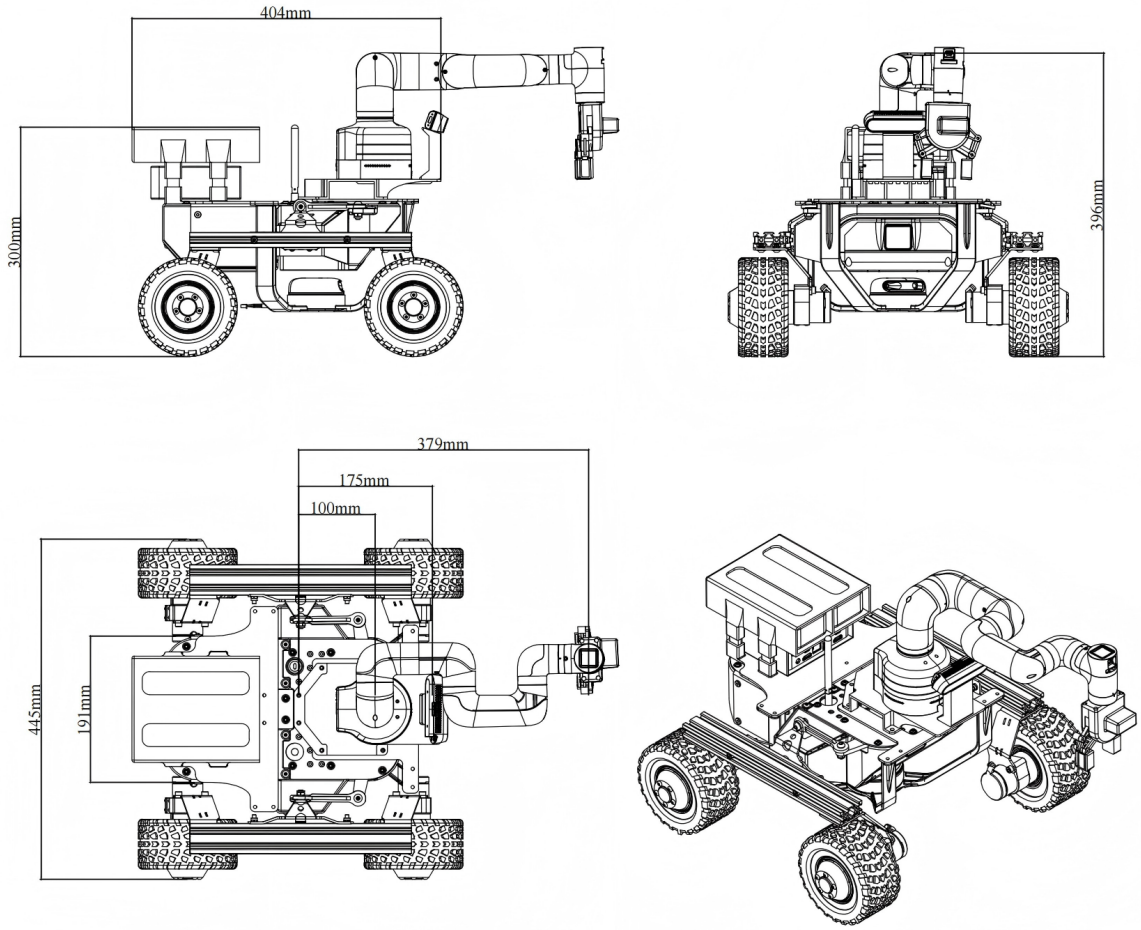


Fig. 5. 3D CAD model of the robot

5.2 Static Stability Analysis

A stability analysis based on the center of gravity was conducted for the robot with the manipulator in its fully extended position, ensuring stability during object grasping and preventing forward tipping. Assuming the CoG of each component is located at its geometric center with uniform mass distribution, the overall CoG position is calculated using the following formula:

$$cg = \frac{\sum(x_i * g_i)}{\sum g_i} \tag{1}$$

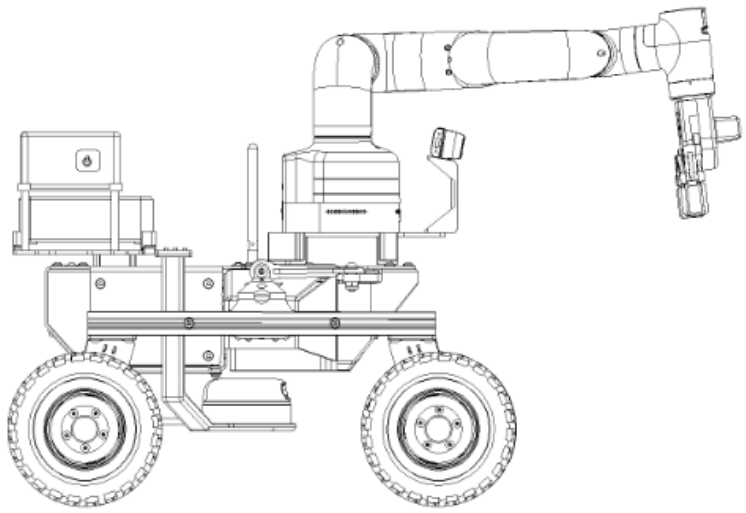


Fig. 6

Using the CoG of the Leo Rover chassis as the coordinate origin (0,0), the mass and positional data of key components are listed in the table below.

Table 1. Data of key components

Component	**Weight (g)**	**Position (x) (mm)**
Leo Rover	6500	0.0
Camera and installation accessories	< 150	173.6
Lidar	190	21.8
NUC and NUC shell	< 1300	-151.6
Manipulator	860	228.1
Gripper	88	388.5
Battery (x2) and battery shell	< 1400	-155.2
Object	< 300	388.5
Total weight	**10788**	-3.46

Substituting the data into the formula yields an overall robot CoG located -3.46 mm of the Leo Rover's center. This position is very close to the central axis, resulting in favorable static stability and providing a robust margin for dynamic operations.

5.2.1 Requirements satisfied:

- **PR18.1:** The total weight of the robot, including all payloads and structural attachments, shall definitely not exceed 18 kg.

5.3 Sensor range

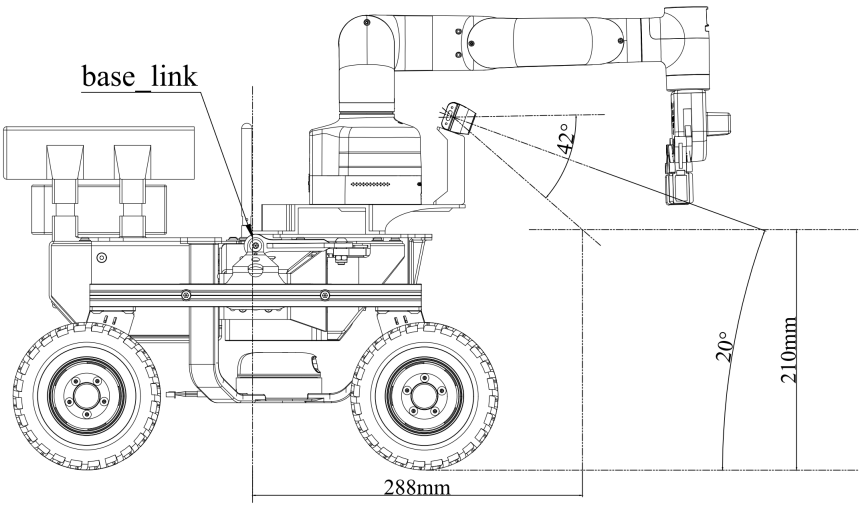


Fig. 7. Camera FOV range

Regarding the camera's detection range, an RGB camera is employed for object recognition. Its vertical FOV is 42°. To meet the minimum detection distance requirement, the camera is tilted downward by 20°, resulting in a minimum detection distance of 288 mm (Compared to base-link). Furthermore, the camera is mounted at the highest feasible position to avoid a co-planar view with the object, thereby obtaining more distinctive, three-dimensional image data for improved recognition.

5.3.1 Requirements satisfied:

- **PR10.1:** The robot shall detect bins at a physical distance greater than 15 cm.

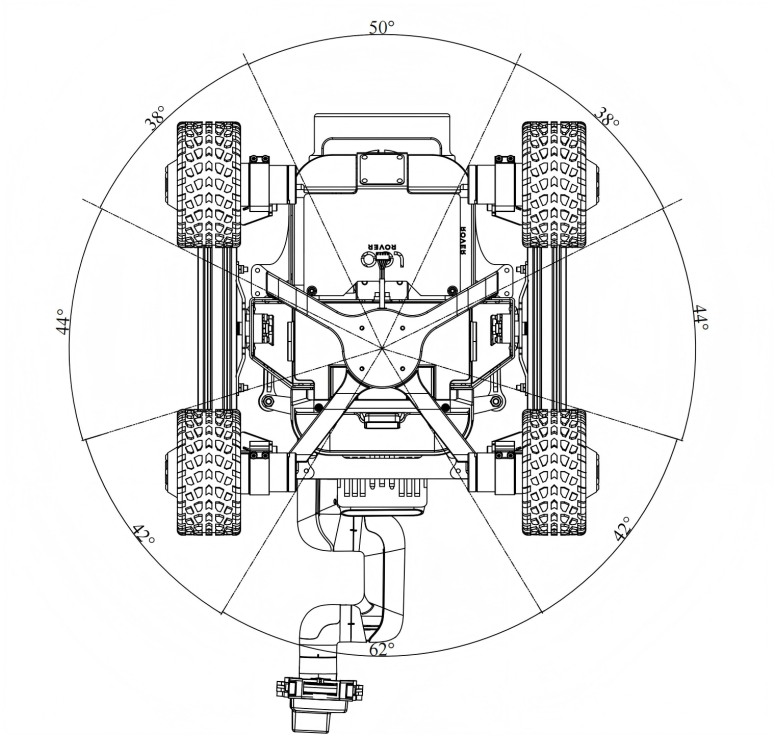


Fig. 8. Radar angle range

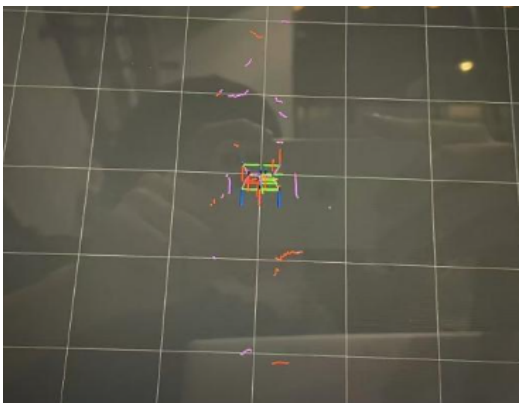


Fig. 9. Actual radar testing

For the LiDAR, given the height constraint, mounting it on top of the rover could lead to occlusion by the front manipulator or the rear NUC, potentially blocking critical obstacle information along the path. Mounting it on the side would sacrifice nearly half of its detection coverage. Therefore, the selected configuration places the LiDAR beneath the rover chassis. This maximizes the unobstructed horizontal scan range. While this choice slightly compromises the robot's off-road clearance, it is acceptable for the intended task which is performed on level ground.

Table 2. Angular coverage at different mounting positions

Mounting Position	Approx. Angular Coverage	Percentage of 360°
Under Chassis	200°	55.5%
Side Mount	140° – 180°	40% – 50%
Top Mount	180° – 220°	50% – 60%

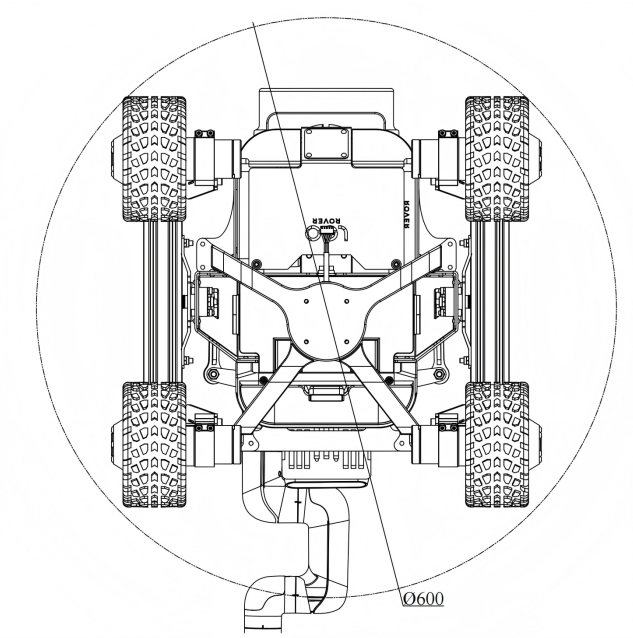


Fig. 10

However, the presence of the rover's wheels within the LiDAR's field of view may interfere with its ability to detect nearby obstacles. This interference creates an area of concern defined by the robot's circumscribed circle centered at the LiDAR, with a radius of 300 mm.

5.3.2 Requirements satisfied:

- **PR5.2:** The system shall detect obstacles within a radial range of 30 cm to 150 cm.

5.4 The grasping range of the robotic arm

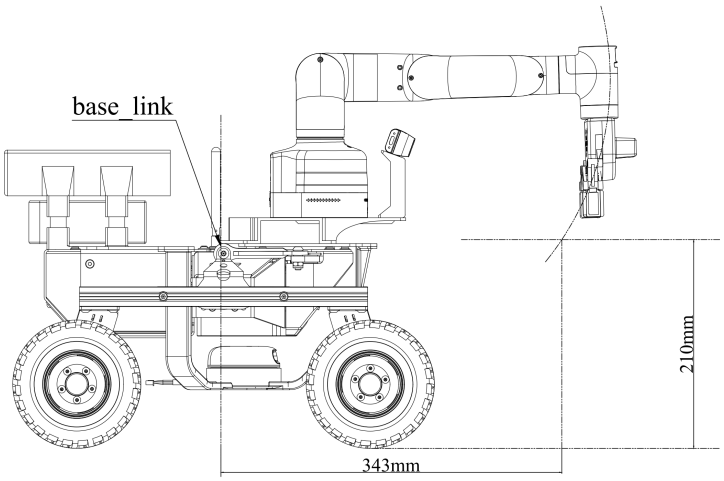


Fig. 11. The working space of the robotic arm

Taking into account the mounting position of the manipulator, potential camera interference, the posture of the end-effector gripper, and the height of the object placement, the estimated grasping distance is approximately 343 mm.

5.4.1 Requirements satisfied:

- **FR11:** The robot shall grasp target objects located within its reachable workspace (within a range of 5 cm to 30 cm).

5.5 Manufacturability

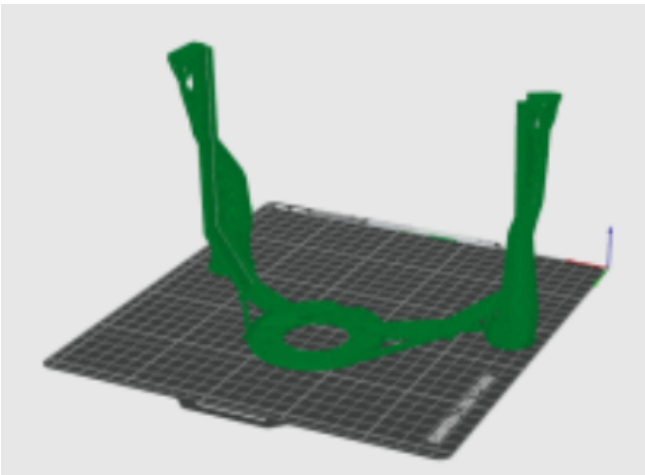


Fig. 12

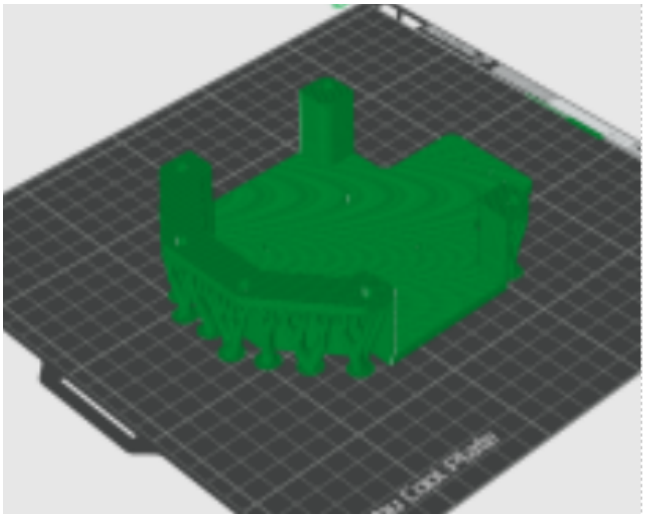


Fig. 13

Key components are manufactured via 3D printing. Considering the inherent limitations of this process, the geometry of these parts is kept as simple as possible. Overhang lengths and inclination angles are minimized to prevent collapse during printing. Furthermore, the size of each individual part is constrained to keep the average print time within 2–3 hours per piece, ensuring time efficiency. Unnecessary structural features are also eliminated to reduce material consumption and cost. Connections between 3D-printed parts are achieved using standard metal fasteners such as screws, nuts, and brass spacers, which are readily available and easy to procure.

5.6 Structural strength and stability

Most components are 3D-printed using TZ PLA Basic material. The specific printing parameters are listed in the table below.

Table 3. 3D Printing Parameter Settings

Parameter	**Value**
Material	PLA Basic
Layer Height	0.2 mm
Infill Density	20%
Infill Pattern	Grid
Effective Tensile Strength	15 – 25 MPa
Safety Factor	5
σ_{safe}	4MPa

Finite element analysis (FEA) was conducted on the primary payload sled, with material properties (PLA) defined as shown in the table below:

Table 4. Finite Element Analysis Parameter Setting Table

Parameter	Value
Density	0.3 g/cm
Elastic Modulus	XY direction: 400 MPa, Z direction: 200 MPa
Poisson's Ratio	0.3

5.6.1 Front payload sled (up)

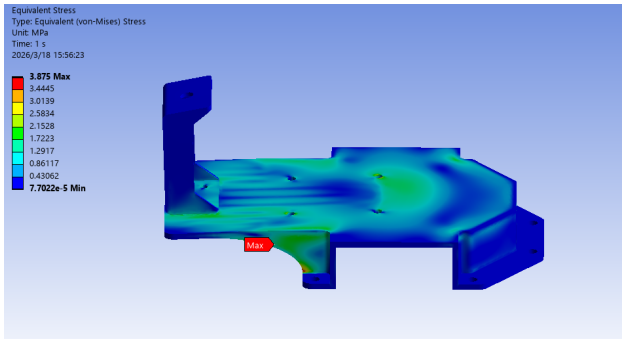


Fig. 14. Stress Distribution Plot

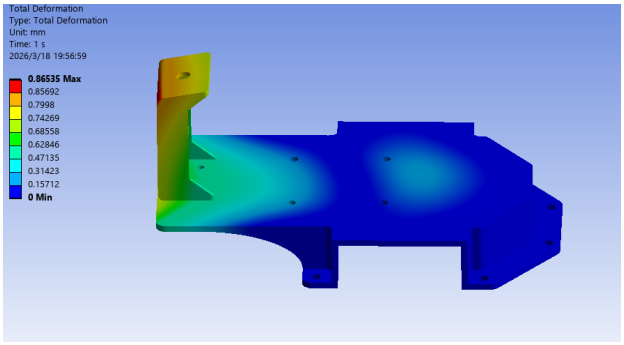


Fig. 15. Deformation Contour

Finite element analysis was performed on the front payload sled (up). The applied loads include the weight of the robotic arm (applied at its center of mass), the combined weight of the gripper and the grasped object (applied at the end effector), and the weight of the depth camera (applied at its mounting location). As shown in the figure, the maximum equivalent stress is 3.875 MPa, which is below the allowable stress σ_{safe} . The deformation at the camera mounting point is 0.8 mm, confirming that the structure has adequate stiffness.

5.6.2 LiDAR bracket

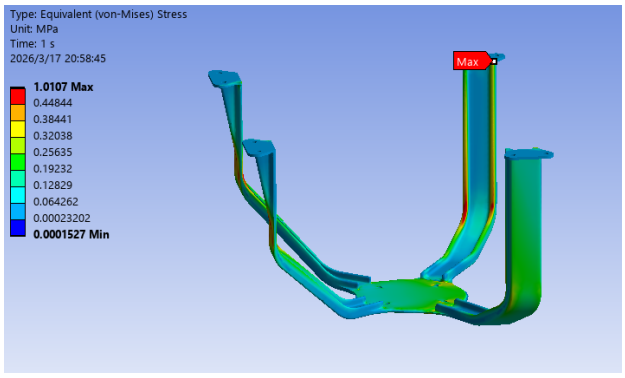


Fig. 16. Stress Distribution Plot

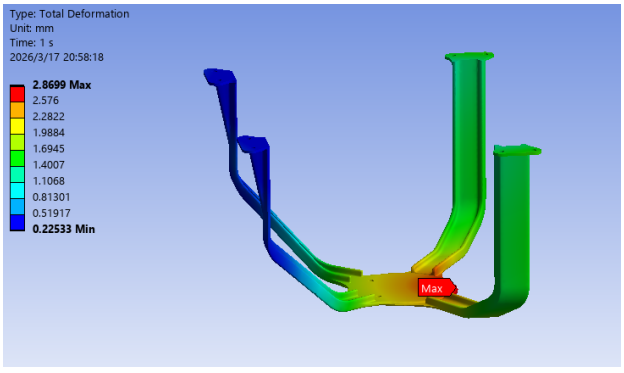


Fig. 17. Deformation Contour

The LiDAR bracket was analyzed as a rigid assembly under the self weight of the LiDAR unit. As illustrated in the figure, the maximum equivalent stress is 1.01 MPa, which remains below the allowable stress σ_{safe}. The deformation at the LiDAR mounting location is 2.87 mm, which corresponds to an angular tilt of 0.7°, both of which are within acceptable limits.

5.6.3 Payload sled for LiDAR mounting

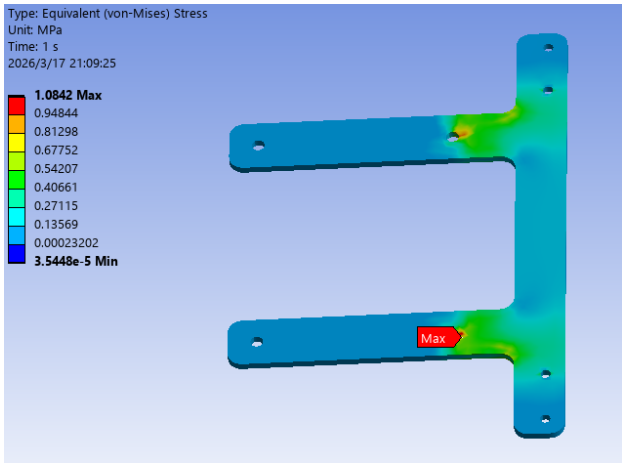


Fig. 18. Stress Distribution Plot

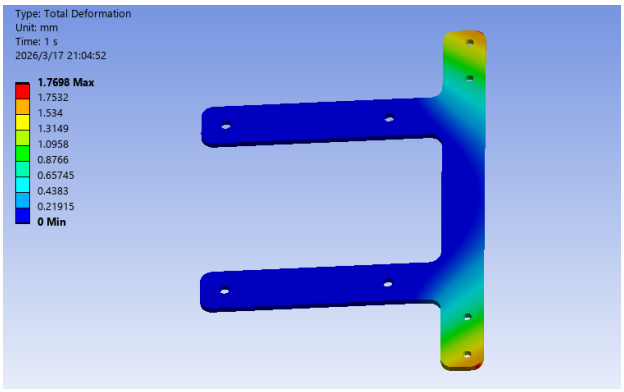


Fig. 19. Deformation Contour

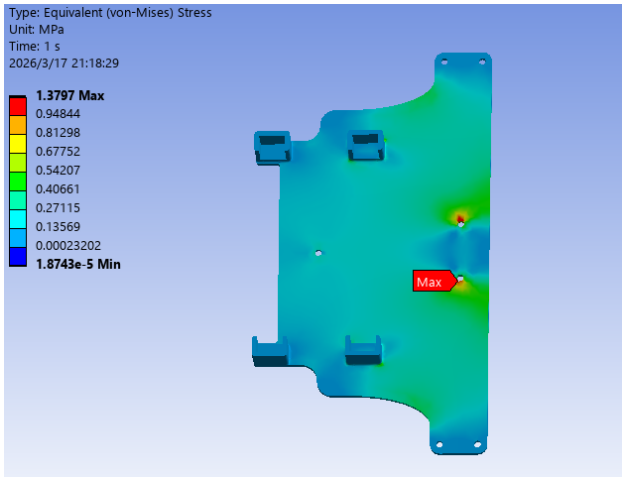


Fig. 20. Stress Distribution Plot

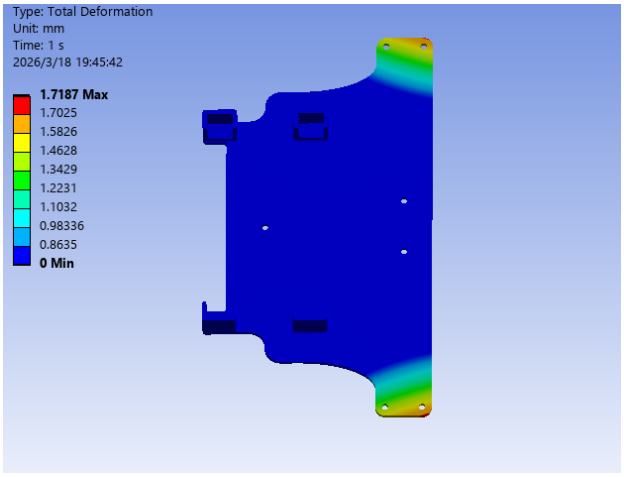


Fig. 21. Deformation Contour

The front payload sled (lower) and the rear payload sled are subjected to loads transmitted from the LiDAR bracket, including the self weight of both the LiDAR unit and the bracket itself. As shown in the figure, the maximum equivalent stress in the front payload sled (lower) is 1.084 MPa with a deformation of 1.77 mm, while the rear sled exhibits a maximum equivalent stress of 1.38 MPa and a deformation of 1.72 mm. In both cases, the maximum stresses remain below the allowable allowable stress σ_{safe}, and the deformations are within acceptable limits.

5.6.4 NUC and battery payload sled

The column T slot that supports the NUC and the battery has been redesigned by removing the original cantilever beam structure and is now entirely 3D printed using PLA material. The cross section of the column measures approximately 6 mm × 10 mm, providing sufficient structural strength to withstand the shear forces generated by the NUC and the battery under low acceleration operating conditions.

6 Electrical Design

6.1 Introduction

This section presents the electrical design of the customised modules, including peripheral power supply, communication interfaces, and the overall power budget. The objective is to verify whether the system can deliver sufficiently stable power to all modules, ensuring that the entire Leo Rover platform operates reliably and completes its designated tasks within the required time frame. The following subsection illustrates the detailed design and associated power ratings using diagrams and icons.

6.2 Power Connection Diagram

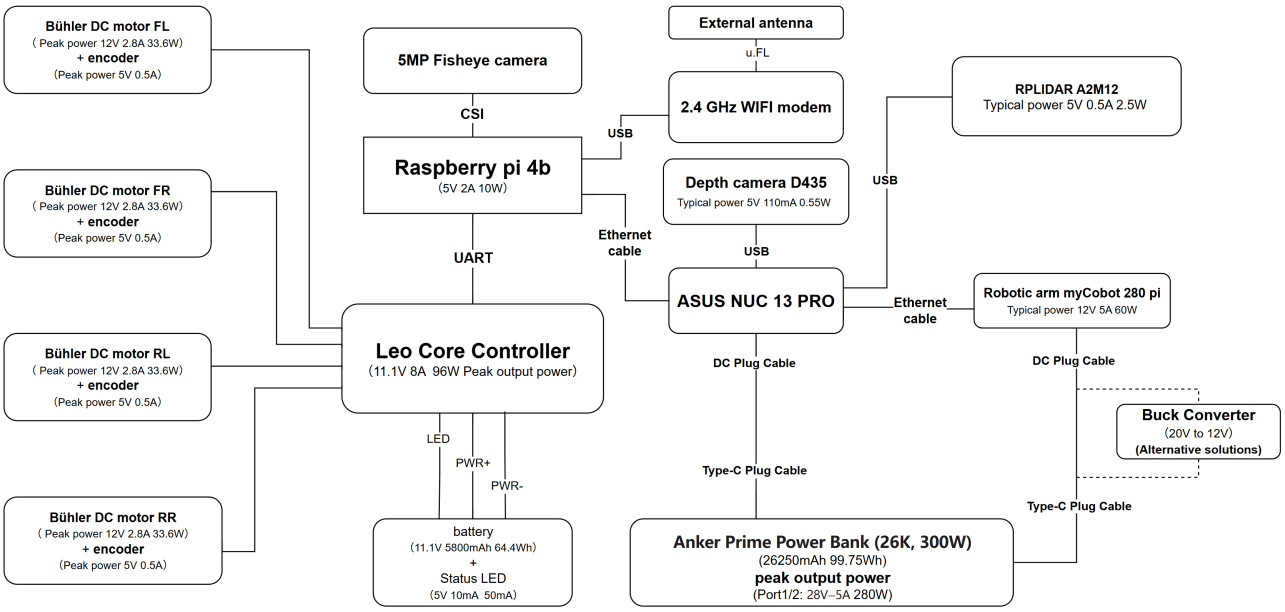


Fig. 22. Power Connection Diagram

The image above provides a visual representation of the overall electrical architecture. On the left side are the components included in the Leo Rover base platform, which consists of four DC motors with encoders, a Raspberry Pi 4B, an STM32 based microcontroller, a monocular camera, an external antenna, and an integrated rechargeable lithium battery. On top of this platform, additional modules such as a stereo depth camera and a robotic arm are required to accomplish the assigned tasks within the specified timeframe.

However, during system operation, the original 5800 mAh battery was no longer capable of supporting additional devices at their required power levels. Therefore, introducing suitable, compact, and

mobile power sources became necessary. To determine the demand, we referred to the specifications of the existing power adapters and the datasheets of the additional hardware required for the mission and calculated the appropriate supply requirements.

Driven by the need for a concise and lightweight setup, and under strict constraints on payload size and weight, we transitioned from traditional bulky DC battery packs to a modern USB PD 3.1 protocol-based solution. We ultimately selected a single Anker Prime Power Bank (26,000 mAh, 300W total output) —— one port dedicated to powering the ASUS NUC and the other for powering the robotic arm. Historically, developers were forced to rely on either heavy camping power stations (which severely reduce rover flexibility and interfere with arm operation) or multiple separate industrial battery packs because compact power banks lacked the massive combined wattage required.

Therefore, this single-battery configuration proves to be an optimal solution by leveraging independent dual USB-C power delivery capabilities. For the computing unit, one Type-C port uses a 20V/5A PD trigger cable to deliver up to 100W directly to the ASUS NUC, completely eliminating the need for the previously required external step-down module in the NUC's power link. Concurrently, the second independent Type-C port is dedicated to the robotic arm. To safely bypass the 3A current limit inherent to the standard 12V PD protocol, an alternative step-down module connection is incorporated into the arm's link. This setup triggers a 20V output from the power bank and converts it down via a buck module, achieving a stable 12V, 5A output to meet the arm's full-load requirement. This highly integrated architecture perfectly achieves our goal: delivering sufficient peak power to both subsystems while drastically reducing system weight, footprint, and cable clutter.

6.3 Power Budget

Peripheral	Required Voltage	Current Draw	Peak Current	Peak Power	Power Source	Notes
Raspberry Pi 4b	5V	2A	3A	15W	Leo core controller	Wi-Fi and remote desktop may increase power
Motor	12V	0.85A	2.8A	33.6W	Leo core controller	Ground slope and payload affect power consumption
LED	5V	0.01A	0.01A	0.05W	Battery	indicate the battery status
Encoders	5V	0.5A	1A	5W	Leo core controller	————————
RPLIDAR	5V	0.6A	2.5A	12.5W	NUC	The power required during startup is relatively high.
ASUS NUC	20V	3A	5A	100W	——	CPU and USB-connected devices affect power consumption.
Camera D435	5V	0.11A	0.7A	3.5W	NUC	Operating mode determines power consumption.
Robotic Arm	12V	3V	5V	60W	20-12V Buck	The robotic arm posture, object mass, and motion complexity determine power consumption.
Leo core controller	11.1V	6A	8A	88.88W	Battery	Power consumption depends on the requirements of the controlled modules.

Fig. 23. Power Budget Table

6.4 Peak Load Calculation And Analysis

6.4.1 powered by Leo rover battery (11.1V 8A)

- Leo core controller: 88.88W
- Raspberry Pi 4B: 15 W
- Encoders:5 W
- LED: 0.05 W
- 4× Motor: 68.83W

Total peak power: 88.88W. This part of the system is powered entirely by the Leo Rover's onboard battery. All peak power outputs are regulated by the rover's internal controller. Given a flat, low-speed, and low-resistance operating environment, the peripheral devices are more likely to reach their peak consumption before the motors do, as the motors rarely operate near their maximum capacity.

Therefore, although the theoretical peak load is 88.88 W, practical operation suggests otherwise: the motors typically draw around 1 A each, and the Raspberry Pi rarely runs at full load. As a result,

the actual estimated power consumption is approximately 60 W, leaving around 20% headroom. Since the battery is rated at 11.1 V and 5.8 Ah, a simple calculation gives a total capacity of 64.4 Wh. Under normal conditions, this capacity is sufficient to support a 30-minute mission.

6.4.2 Anker Prime Power Bank (26K, 300W) port 1 (28V 5A)

- ASUS NUC: 100 W
- RPLIDAR: 12.5 W
- Camera D435: 3.5 W

Total peak power: 100W. This subsystem is powered by a dedicated USB-C port on the Anker Prime Power Bank. Utilizing a 20V PD trigger cable, the power bank directly supplies the NUC with a stable voltage and current, completely eliminating the need for the previously used DC-DC buck regulator (SC200-3648V20XX). Both the LiDAR and depth camera are powered and communicate directly through the NUC's USB ports. Therefore, the peak consumption of this segment is fundamentally determined by the NUC itself. The selected Type-C port can deliver up to 140 W of output power, meaning that even at peak demand, the system remains sufficiently powered to ensure stable operation of the "central control computer." Actual consumption depends on CPU utilisation and peripheral load, and is typically estimated around 60 W, leaving ample headroom. With a shared total energy capacity of 99.8 Wh (26,000 mAh), this single battery is capable of efficiently satisfying the mission runtime requirements for the entire system.

6.4.3 Anker Prime Power Bank (26K, 300W) port 2 (28V 5A)

- Robotic Arm: 60 W

Total peak power: 60W. This subsystem shares the same highly capable Anker Prime Power Bank via its second independent USB-C port (Port 2). In the default configuration, the robotic arm is directly connected to the power bank using a standard 12V PD trigger cable to maximize system lightweighting. However, bounded by the native 12V USB PD protocol's standard 3 A (36W) output limit, a conditional upgrade strategy is incorporated into the system design: the installation of an external DC-DC buck regulator will only be considered if obvious power deficiency or overcurrent disconnection occurs during actual operation testing. If the direct connection allows the arm to operate normally, the regulator will be omitted. In the event of insufficient power, the link will be upgraded to a 20V PD trigger configuration, utilizing the regulator to step the voltage down from 20 V to 12 V,

thereby safely unlocking a stable 5 A (60W) full-load output. It is straightforward to conclude that this flexible, tiered power strategy not only maintains exceptional integration under typical workloads but also reserves ample upgrade headroom for peak demands, reliably supporting the robotic arm in completing its assigned tasks within the required timeframe.

To summarise, aggregating the peak power requirements yields 88.88 W + 100 W + 60 W = 248.88 W. From the tabulated calculations, the expected power consumption in regular operation is approximately 156.5 W.

6.5 Analysis of the Electrical Design Requirements

6.5.1 PR15.1:

During chassis-loaded motion and full-load computation and operation, including RPLiDAR, cameras, and robotic arm operation, the power system can support a 30-minute task duration, as demonstrated by the calculations above.

6.5.2 FR3:

The Leo Rover, NUC, and robotic arm are interconnected via Ethernet. These three cables are connected through a 1-to-2 splitter that provides basic switching functionality. Each device has an independent link, and frames are deterministically forwarded by the internal switching structure. Therefore, as long as the physical cables and connectors remain intact, the theoretical packet loss rate in such a small, closed local network is effectively 0%. This satisfies the design requirement.

6.5.3 FR19:

The customized electrical subsystem uses insulated wiring, DC5525/DC5521 connectors, high current DC-DC buck regulator, type-C to DC cable and connecting terminal blocks rated above 10 A. This ensures that the battery and power distribution system maintain proper shielding and electrical isolation.

6.5.4 FR20/21:

All three batteries are equipped with independent physical power switches, enabling rapid shutdown in emergency situations. Additionally, a software-level emergency stop mechanism is implemented to further ensure system safety during unexpected events.

7 Software Design

The **GitHub** repository for this project is available at:
`https://github.com/IvaJorgusheska/AERO62520-Robotic-Systems-Design-Project`.

7.1 Software Architecture Overview

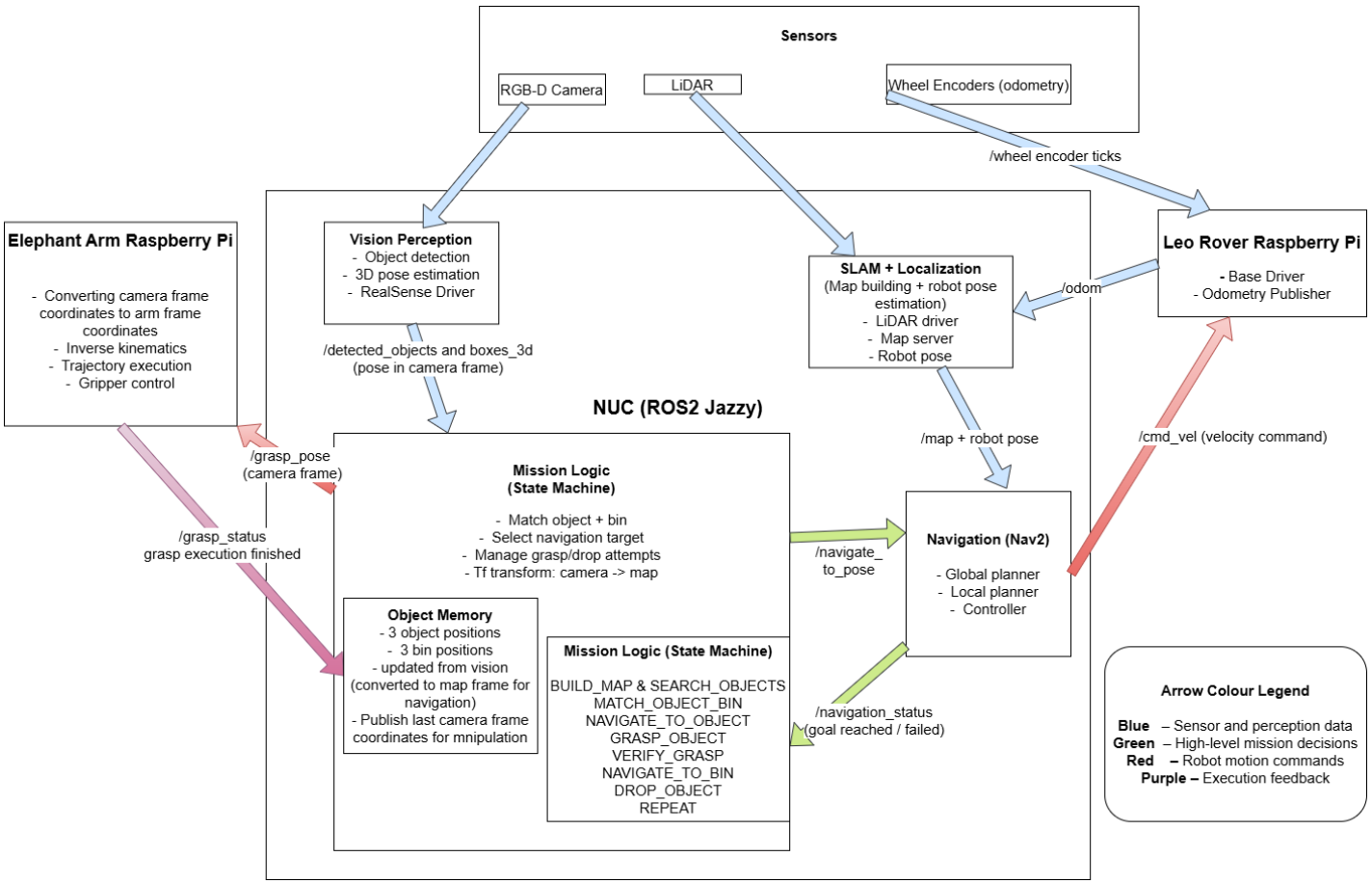


Fig. 24. Software architecture block diagram.

The software architecture illustrated in the block diagram integrates perception, localisation, navigation, mission planning, manipulation planning, and low-level actuation into a distributed ROS2 Jazzy system. The Intel NUC performs all computationally intensive tasks including SLAM, YOLOv8 perception, Nav2 path planning, and mission-level decision making. Meanwhile, the Leo Rover onboard Raspberry Pi and the myCobot 280 Pi execute the low-level control of the mobile base and manipulator respectively.

Communication between these computers is handled through the ROS2 DDS middleware, allowing deterministic exchange of sensor data, transforms, and control commands. This distributed architecture enables the robot to autonomously complete the full **search–retrieve–transport–deposit**

mission cycle after a single start command, satisfying the autonomous operation requirement (FR1).

The system is organised into five major functional pipelines that interact throughout the mission:

1. **LiDAR-based SLAM** for localisation and mapping
2. **RGB-D perception** for object and bin detection
3. **Mission planning** through a finite state machine
4. **Nav2** for global and local path planning
5. **Manipulation planning and execution** for grasping and depositing objects

Each pipeline is assigned to the computational unit best suited to its processing requirements while maintaining clear separation between perception, decision-making, and actuation layers.

7.2 Mission Planning and Task Coordination

At the centre of the autonomy architecture lies the **mission planner**, implemented as a finite state machine running on the Intel NUC. The mission planner coordinates perception, navigation, and manipulation subsystems in order to complete the full **search–pick–place mission cycle.**

During operation the robot continuously explores the environment while simultaneously building a map using SLAM and detecting objects and bins using the RGB-D perception system. All detections are initially expressed in the **camera frame** and are immediately transformed into the **global map frame** through the TF tree.

The mission planner maintains an internal **semantic memory structure,** implemented as a hash map that stores the most recent known locations of detected targets. Because the system recognises three colours, the memory contains six entries:

- three **object locations**
- three **bin locations**

Each object and bin location is stored in the **map frame**, allowing them to be used directly as navigation goals.

As the robot explores the environment, the perception system continuously publishes object detections. Whenever an object or bin is detected, its position is stored or updated in the memory map.

Once both an **object and a bin of the same colour** have been detected, the mission planner proceeds to the next phase of the task.

Mission execution then follows a sequence of states:

1. Exploration and Mapping

The robot navigates through the environment while SLAM builds a global map and the vision system searches for coloured objects and bins. Detected targets are continuously stored in the semantic memory structure.

2. Object–Bin Pair Selection

When the system identifies a valid pair consisting of an object and a bin with the same colour, the mission planner selects the object as the next target and transitions to the navigation phase.

3. Navigation to Object

The stored object position in the **map frame** is sent to the Nav2 navigation stack, which generates a collision-free path and drives the rover to a position within the manipulator's reachable workspace. Nav2 communicates goal completion to the mission planner through the result of the `NavigateToPose` action interface. Once the action returns a successful result, the mission planner transitions to the grasp execution state.

4. Grasp Execution

Once Nav2 reports that the goal position has been reached, the mission planner sends the latest object coordinates in the **camera frame** to the manipulator controller. The arm then converts the coordinates to its arm frame and performs inverse kinematics. It opens the gripper and executes the grasping motion.

After the grasp attempt, the arm returns to its initial pose while keeping the gripper closed.

5. Grasp Verification and Retry

The mission planner verifies whether the grasp was successful by checking if the camera still detects the object at the same location. Because the vision system continuously publishes detections, the planner can determine whether the object remains visible.

If the object is still detected, the planner assumes the grasp attempt failed and sends another grasp command to the manipulator. This retry behaviour ensures reliable object retrieval and supports the grasp success requirement (FR12, PR12.1).

6. Navigation to Bin

Once the object has been successfully grasped, the planner retrieves the stored bin position of the corresponding colour from the semantic memory and sends this location as a navigation goal.

The rover then navigates autonomously to the bin using Nav2.

7. Object Placement

When the rover reaches the bin location, the planner sends the bin coordinates in the camera frame to the manipulator. The arm moves to the target position and releases the object by opening the gripper, completing the deposit operation.

8. Mission Continuation

After the object has been successfully deposited, the arm returns to its initial configuration and the mission planner resumes exploration to locate the remaining object–bin pairs. This cycle continues until all three objects have been successfully collected and deposited.

This state-machine-based mission planner ensures coordinated interaction between perception, navigation, and manipulation subsystems while enabling robust retry behaviour and autonomous completion of the full mission cycle.

The interactions between perception, mission planning, navigation, and manipulation described above are illustrated in the software architecture block diagram shown in Fig. 24. The diagram highlights the flow of sensor data, mission decisions, and control commands between system components, as well as the coordinate frame transformations between the camera frame and the global map frame used for navigation and manipulation.

7.3 LiDAR-Based SLAM and Localisation

The robot operates in a previously unknown indoor environment and therefore requires simultaneous localisation and mapping (SLAM) to build a map while estimating its own pose. This functionality is implemented using the ROS2 SLAM package **slam_toolbox**, running on the Intel NUC. The SLAM system fuses two primary sensor inputs:

- 360° laser scans from the **A2M12 LiDAR**
- wheel odometry from the **Leo Rover Pi**

The LiDAR provides dense geometric measurements of the environment at **10 Hz**, while wheel odometry supplies continuous motion estimates between scans. Their combination enables robust localisation: LiDAR provides accurate spatial constraints for scan matching, while odometry stabilises pose estimation during robot motion.

A LiDAR-based SLAM approach was selected instead of visual SLAM because the test environment consists mainly of large white cardboard walls and obstacles with well-defined planar surfaces. These produce consistent LiDAR returns and reliable scan matching, whereas visual SLAM would be more sensitive to lighting variation, low texture, and motion blur. In addition, the RGB-D camera is already dedicated to object detection, so separating mapping and perception across different sensors improves overall system robustness.

To improve scan reliability, a dedicated **LiDAR filtering node** is included in the sensing pipeline. During testing, reflections from the rover's wheels occasionally produced erroneous measurements that degraded SLAM performance. To mitigate this, LiDAR data within the angular ranges corresponding to the wheel positions are removed before further processing. The filtered scans are published on `/scan_filtered`, and all downstream subscribers, including the SLAM module and navigation stack, use this topic instead of the raw scan. This preprocessing reduces noise and improves map consistency.

The SLAM system operates in **online asynchronous mode**, prioritising the most recent sensor frames. This sacrifices a small amount of map optimisation accuracy in favour of improved real-time performance, allowing SLAM to run concurrently with other computationally demanding components such as object detection and navigation.

The generated occupancy grid map uses a spatial resolution of **5 cm** $\times$ **5 cm**, which provides sufficient detail to represent obstacles and navigable corridors while remaining computationally efficient. The SLAM module also provides the transform tree used throughout the autonomy stack. The primary coordinate frames are:

- `map` – global reference frame generated by SLAM
- `odom` – local odometry reference frame
- `base_link` – robot base frame
- `camera_link` – RGB-D camera frame used for object detection

These form the transform chain:

$$\texttt{map} \rightarrow \texttt{odom} \rightarrow \texttt{base\_link} \rightarrow \texttt{camera\_link}$$

This transform structure allows object detections in the camera frame to be converted into global coordinates for navigation and task planning.

Running SLAM on the rover's onboard Raspberry Pi was rejected because it could not sustain real-time pose graph optimisation alongside its other required processes. The SLAM node therefore runs on the NUC, which provides sufficient computational resources for simultaneous mapping, localisation, object detection, and navigation. This configuration enables localisation accuracy within $\leq$**10 cm (PR4.2)** and limits long-term mapping drift to $<$**10 cm (PR4.1)**.

The SLAM outputs are consumed by several subsystems in the autonomy stack. The map and pose estimates are used by the Nav2 navigation system for path planning and obstacle avoidance **(FR4, FR7, FR8)**. The mission logic uses the transform tree to convert detected object poses from the camera frame into global coordinates **(FR7)**. Accurate localisation also ensures that the robot can position itself within the manipulator workspace required for grasping **(FR6)**.

Overall, the SLAM subsystem satisfies the localisation requirement **FR4** and **PR4.2**, supports obstacle detection and environment mapping in accordance with **FR5** and **PR5.1–PR5.3**, and forms the basis of the autonomous navigation behaviours required by **FR4**. By providing a consistent global reference frame for perception, navigation, and manipulation, it ensures coherent operation across the full robotic system.

7.4 Navigation and Path Planning (Nav2)

Autonomous navigation is implemented using the ROS2 **Navigation2 (Nav2)** framework running on the Intel NUC. Nav2 integrates localisation information provided by SLAM, layered costmaps derived from LiDAR observations, and navigation goals generated by the mission control system.

Exploration of the unknown environment is performed using the **explore-lite** frontier exploration package. This module analyses the occupancy grid map generated by SLAM and identifies frontier regions representing boundaries between explored and unexplored space. The exploration node generates navigation goals at these frontier locations and sends them to Nav2 using the `NavigateToPose` interface. Nav2 then performs the detailed path planning and motion execution through its internal behaviour tree. By repeatedly navigating to frontier points, the robot gradually explores the environment while building the map and searching for target objects and bins.

Global path planning within Nav2 is performed using the **Smac Hybrid-A* planner**

(`nav2_smac_planner/SmacPlannerHybrid`). This planner was selected as an improvement over the standard NavFn planner due to its ability to generate **kinematically feasible and smooth trajectories** for non-holonomic platforms such as the Leo Rover.

Unlike grid-based planners such as NavFn, which produce piecewise-linear paths with sharp angular transitions, the Smac Hybrid-A* planner incorporates motion constraints and searches in a continuous state space. This results in paths with **continuous curvature and reduced heading discontinuities**, leading to smoother navigation behaviour. This is particularly beneficial in indoor environments with constrained spaces, as it reduces abrupt turns, improves path tracking performance, and enables more accurate alignment when approaching target objects for manipulation. These characteristics directly support the system-level autonomous navigation requirement defined in **FR4**.

Local motion control is implemented using the **Model Predictive Path Integral (MPPI) controller** (`nav2_mppi_controller::MPPIController`). This controller replaces previously evaluated approaches such as the Regulated Pure Pursuit (RPP) and Dynamic Window Approach (DWB) controllers, providing a more advanced control strategy based on stochastic trajectory optimisation.

The MPPI controller operates by sampling multiple candidate control trajectories over a finite time horizon and evaluating them using a cost function that considers path tracking error, obstacle proximity, and control effort. The optimal control command is selected based on the lowest-cost trajectory. This results in **smoother velocity profiles**, improved obstacle avoidance behaviour, and more stable path tracking compared to purely geometric or reactive controllers.

The combination of the Smac Hybrid-A* global planner and MPPI local controller produces a **consistent and smooth navigation pipeline**, where curvature-aware global paths are effectively tracked by a predictive controller. This reduces oscillatory behaviour, minimises sharp directional changes, and improves overall motion stability, particularly during object approach and final positioning.

A key trade-off of the MPPI controller is its **increased computational demand**. Due to the need to sample and evaluate multiple trajectories at each control step, MPPI introduces higher CPU usage compared to simpler controllers such as RPP or DWB. This additional load is handled by the Intel NUC, which provides sufficient computational resources to run navigation alongside SLAM and perception. Despite the higher computational cost, the improved motion quality and robustness were prioritised to ensure reliable task execution.

Both global and local **costmaps** incorporate LiDAR-derived obstacle data, inflation layers, and the robot footprint model. These layers allow the navigation system to represent obstacles with appropriate safety margins and enable reliable detection of obstacles in the required 30–150 cm size

range **(PR5.2)** while ensuring that obstacle boundaries are captured with more than 80% coverage **(FR5)**.

The costmap configuration remains fixed during operation. Instead of dynamically modifying parameters such as the inflation radius or velocity limits, the system calculates navigation goals that include a predefined **standoff distance** from the detected object. The mission controller computes a target position offset from the object's location such that the rover stops at a safe distance while keeping the object visible to both the RGB-D camera and the manipulator. This approach ensures that the robot halts approximately **30–35 cm from the target (PR7.1)**, positioning the object within the manipulator workspace required for reliable grasping **(FR6)**.

The Nav2 behaviour tree manages goal execution, path replanning, and recovery behaviours such as costmap clearing when navigation failures occur. These mechanisms increase system robustness and ensure that the robot can reliably approach detected objects **(FR7)**, and navigate to the corresponding bin locations **(FR8)**. Reliable navigation and automatic recovery behaviours also contribute to completing the full mission within the required time limit of **20 minutes (PR1.2)**.

Overall, the Nav2 subsystem provides the complete autonomous navigation capability required for the mission. By combining frontier-based exploration, curvature-aware global path planning, predictive local motion control, and costmap-based obstacle avoidance, the navigation stack fulfils the system-level navigation requirement defined in **FR4** while ensuring safe operation within the rover's hardware constraints **(FR17)**.

7.5 RGB-D Vision: Object and Bin Detection

The robot uses an **Intel RealSense RGB-D camera** streaming to the Intel NUC. The perception pipeline runs continuously at **10 Hz**, processing RGB-D data to detect objects and bins throughout the mission. Unlike a reactive approach, detections do not immediately trigger navigation. Instead, all observations are continuously published and stored by the mission planner, which maintains a semantic memory structure. The robot only transitions to target approach once a valid **object–bin pair of matching colour** has been identified.

The perception node publishes two topics at **10 Hz**:

- the **3D positions** of all detected objects and bins
- a **binary detection flag** indicating whether any valid detection is present

This continuous publishing strategy ensures that the mission planner always has access to the latest observations. While this increases computational load, it simplifies system logic and enables additional functionality such as grasp verification, where persistent detections indicate grasp failure.

7.5.1 YOLOv8s Detector and Training

YOLOv8s was selected due to its ability to run in real time on the NUC CPU without GPU acceleration, making it suitable for continuous onboard inference alongside SLAM and navigation.

The model was **fine-tuned on a custom dataset** collected during development, consisting of images of the target objects and bins under the expected operating conditions. This improves detection robustness compared to using a pre-trained model alone, ensuring reliable performance within the specific environment.

YOLOv8s performs reliably on small indoor objects at short ranges, supporting **PR9.1** by enabling accurate detection at range 20–60 cm. Its single-stage architecture ensures low-latency predictions, allowing the system to maintain real-time perception at 10 Hz.

7.5.2 Class Label Strategy

The detector is trained on six classes:

- red object, yellow object, purple object
- red bin, yellow bin, purple bin

This class design encodes both **semantic type (object/bin)** and **colour** directly into the label. This decision was made because the task depends entirely on colour-based matching rather than object shape **(FR9, FR10)**.

Embedding this information into the class labels eliminates the need for a separate classification stage and enables immediate association between objects and their corresponding bins. This significantly simplifies the mission logic and directly supports the object–bin pairing requirement **(FR14)**.

7.5.3 3D Localisation and Design Choices

Three-dimensional localisation is obtained by combining YOLO detections with depth data from the RealSense camera. For each detection, the corresponding depth region is filtered using a median operator, and the centre of the bounding box is projected into 3D space using camera intrinsics.

Only the **3D centre position** of each object is estimated, rather than a full 6D pose. This design choice is justified by the task requirements:

- the objects are simple and do not require orientation-aware grasping
- the manipulator requires only a central grasp point
- estimating orientation would increase computational cost without improving performance

This simplified representation reduces computational load while remaining sufficient for both navigation and manipulation.

The perception pipeline provides reliable 3D position estimates within the required detection range of **20–60 cm**. The estimated positions are used by:

- the navigation system (after transformation into the map frame) to compute approach goals
- the manipulator, which independently transforms the coordinates into its own reference frame for grasp execution

Bin detection is performed within the same pipeline, ensuring consistent classification and enabling correct object–bin assignment for deposition **(FR14)**. Continuous perception also supports grasp validation, as persistent detections after a grasp attempt indicate failure, enabling retry behaviour **(FR12, PR12.1)**.

Overall, the RGB-D vision subsystem satisfies object and bin detection requirements **(FR9, FR10, PR9.1–PR9.2, PR10.1–PR10.2)** and enables correct colour-based task execution required for successful object deposition **(FR14)**. By continuously supplying perception data to the mission planner, it supports reliable decision-making and contributes to efficient completion of the mission within the time constraint **(PR1.2)**.

7.6 Manipulation Coordination on the NUC

The Intel NUC performs the high-level coordination of manipulation tasks as part of the mission state machine. Once the robot reaches the final approach position near an object, the NUC provides the manipulator subsystem with the information required to execute a grasp.

The perception pipeline publishes the detected object position in the `camera_link` frame. The mission logic then forwards this coordinate together with a manipulation command to the arm subsystem. The commands transmitted to the arm controller include:

- the detected object position in the camera frame
- commands to `open` or `close` the gripper
- commands to move the arm to predefined poses (e.g. `initial` or `transport` positions)

This architecture keeps the NUC responsible only for high-level task planning and perception integration, while the detailed motion planning and execution are delegated to the arm's onboard controller.

A key architectural feature is that the same **3D object position** obtained from the RGB-D perception pipeline is used both for navigation and manipulation. Nav2 uses the object position to compute the final approach pose, while the arm controller uses the same coordinate to compute the grasp pose. This shared representation ensures consistent spatial reasoning between navigation and manipulation and supports the requirement that the robot positions itself correctly for grasping **(FR6)**.

The camera is used to check if the grasp was successful, if a failure occurs, the mission controller can initiate a retry sequence, supporting the grasp retry behaviour defined in **FR12** and **PR12.1**.

7.7 Manipulator Control on the Arm Raspberry Pi

All low-level manipulator control is executed locally on the Raspberry Pi integrated in the **myCobot 280 Pi**. This controller receives high-level commands from the NUC and performs the necessary coordinate transformations, inverse kinematics calculations, and motor commands required to execute the manipulation task.

The arm controller first transforms the received object position from the `camera_link` frame into the arm's coordinate frame. Using this transformed pose, the controller computes the joint configuration required to reach the target position and executes the motion using the **pymycobot** control library.

The **pymycobot** library was selected because it provides direct and lightweight control of the manipulator joints and gripper while requiring minimal computational resources. For the grasping tasks required in this system, the library provides sufficient functionality to perform reliable pick-and-place operations without the overhead of a full motion planning framework.

During development, the **MoveIt2** manipulation framework was also evaluated. Although MoveIt2 provides advanced motion planning capabilities, it was found to introduce significant computational overhead when running on the NUC without providing a clear advantage for the relatively simple grasping tasks required in this project. As a result, the lightweight pymycobot-based control approach was selected instead.

For grasp execution, the end-effector orientation parameters (roll, pitch, and yaw) are fixed to predefined values that were determined experimentally. These values produce a stable **elbow-up configuration** that provides a safe approach trajectory for picking up objects located in front of the robot. Because the target objects have simple shapes and are placed in unobstructed areas, a fixed orientation approach is sufficient for reliable grasping while significantly simplifying the inverse kinematics computation.

Additionally, the manipulator operates in a configuration where the arm motion does not intersect with other components of the robot such as the camera or base structure. This eliminates the need for complex collision checking and ensures safe operation during pick-and-place actions.

Once the target joint configuration is reached, the controller executes the gripper command to close and secure the object. After grasping, the arm moves to a transport pose before navigating to the corresponding bin location, where the gripper opens to release the object. All of the commands, for opening/closing the gripper, and moving the manipulator to target or initial position is determined by messages on specific topics published by the NUC.

This manipulation subsystem therefore enables the robot to perform the required pick-and-place behaviour defined by **FR11**, ensures stable grasping during transport **(FR13)**, supports retry behaviour when grasping fails **(FR12)**, and allows correct deposition of objects into the designated bins **(FR14)**.

7.8 Base Control (Leo Rover Raspberry Pi)

The Leo Rover Raspberry Pi executes all low-level motor control, ensuring stable and responsive base motion independent of the computational load on the NUC. It receives velocity commands from Nav2 via `/cmd_vel` and converts them into motor PWM signals driving the rover's wheels.

The controller also publishes wheel odometry required by SLAM and Nav2, contributing to the localisation requirement **FR4**. In addition, it enforces the rover's velocity limits specified in **FR17** and handles rapid stop commands to satisfy the safety requirements **FR20** and **FR21**.

Reliable base control enables precise approach behaviour within the required **30–35 cm range (PR7.1)**, allowing the manipulator to reach objects **(FR6)** and supporting navigation to the correct bin **(FR8)**.

7.9 Summary and Analysis of Software Compliance

The final software architecture integrates SLAM, perception, navigation, manipulation, and real-time control into a distributed ROS2 system running across the Intel NUC, the Leo Rover Raspberry Pi, and the myCobot Raspberry Pi. High-level perception, navigation, and mission coordination run on the NUC, while the rover and arm controllers handle low-level motion and manipulation execution.

Requirement verification is addressed throughout the subsystem descriptions. Localisation and mapping satisfy **FR4** and associated performance requirements, while the RGB-D perception system provides object and bin detection and classification in accordance with **FR9** and **FR10**. Navigation through the Nav2 framework fulfils **FR7–FR8** and contributes to the overall navigation capability defined in **FR4**.

Manipulation using the myCobot controller satisfies **FR11–FR14**, including grasping, holding, retry behaviour, and correct object deposition. Mission progression and full task completion comply with the autonomous operation and timing requirements **(FR1, PR1.2)**. Safety mechanisms implemented across the system ensure compliance with **FR20** and **FR21**.

Overall, the architecture provides a modular and efficient autonomy stack capable of completing the required mission tasks.

7.10 RQT Graph and System Integration

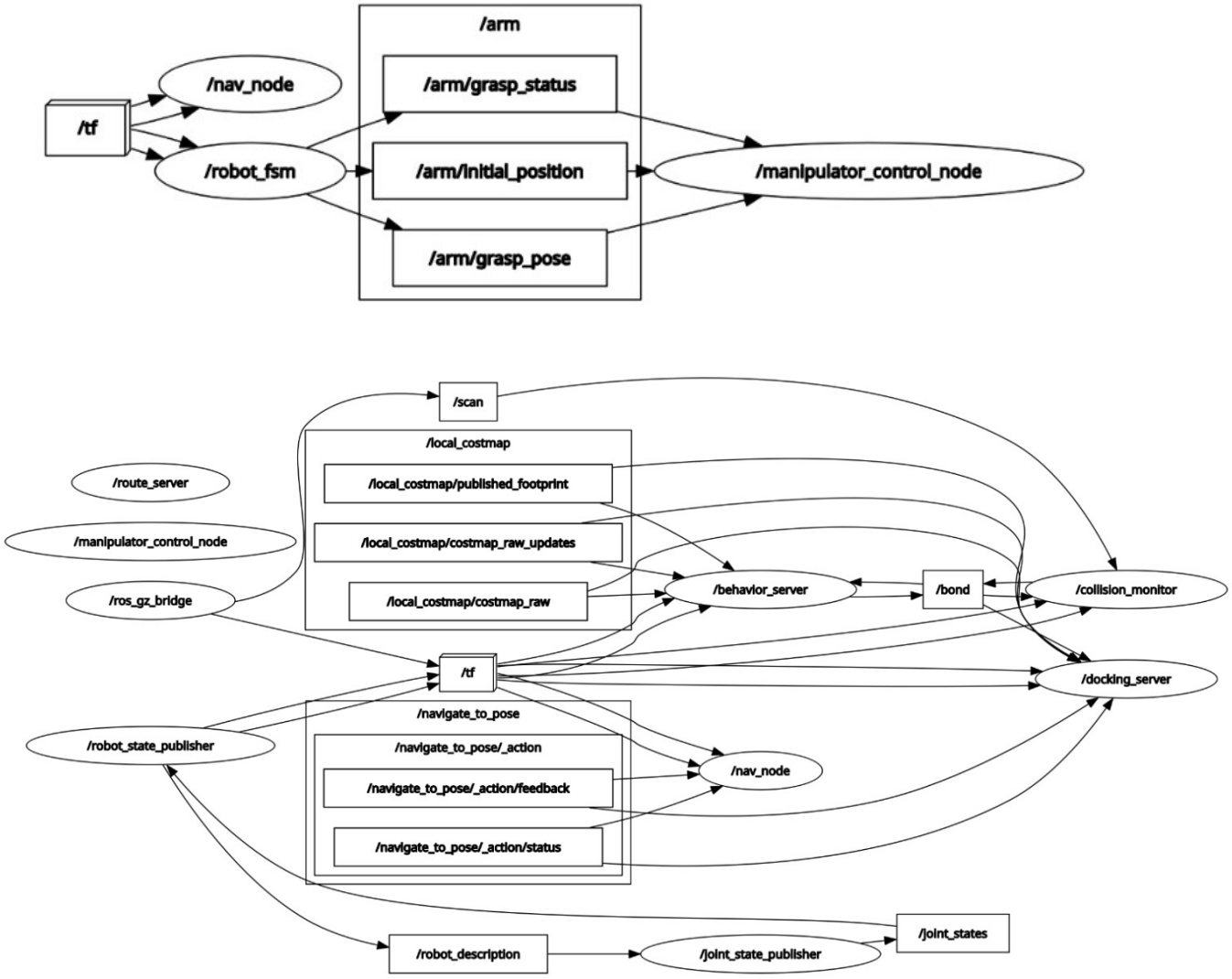


Fig. 25. RQT graph showing node and topic interactions for the implemented system.

The RQT graphs are presented in two parts, corresponding to the two main execution pipelines: (i) perception–manipulation and (ii) navigation. This separation reflects the system architecture shown in Fig. 3, where computation is performed on the NUC, while actuation is executed on the arm and rover Raspberry Pi units, with all inter-device communication occurring over Ethernet via the hub.

In the perception–manipulation pipeline, the Intel RealSense camera, connected via USB to the NUC, publishes `/detected_object` and `/detection_state` at approximately **10 Hz**. These represent the 3D position of detected objects (in the camera frame) and the presence of a valid detection. Both topics are consumed by the mission controller (`/robot_fsm`), which determines when manipulation should be triggered.

When a grasp action is required, the mission controller publishes `/arm/grasp_pose`, which speci-

fies the **target object position in the camera frame**. In addition, `/arm/grasp_status` is used as a command signal (open/close gripper), and `/arm/initial_position` commands predefined arm configurations. These topics are transmitted over Ethernet via the hub to the arm Raspberry Pi, where the `/manipulator_control_node` performs the transformation to the arm frame, computes inverse kinematics, and executes the motion.

In the navigation pipeline, the RPLIDAR A2M12 (connected via USB to the NUC) publishes `/scan`, which is used by the Nav2 stack together with TF and odometry. The navigation node (`/nav_node`) interfaces with Nav2 through the `/navigate_to_pose` action, with status and feedback topics indicating goal execution. The resulting velocity commands are published as `/cmd_vel` and executed by the rover Raspberry Pi. This pipeline operates at approximately **10 Hz**, providing stable control of the rover base.

The RQT graphs therefore directly reflect the system block diagram: sensors (camera and LiDAR) are connected via USB to the NUC, all high-level computation (perception, mission logic, navigation) runs on the NUC, and commands are sent over Ethernet via the hub to the arm and rover Raspberry Pi controllers, which execute the corresponding actions. No direct communication exists between navigation and manipulation subsystems; all coordination is handled centrally by the mission controller.

Overall, the observed topic structure confirms correct integration of sensing, decision-making, and actuation across the distributed system architecture.

8 Analysis

Rather than presenting the Requirements Verification Matrix (RVM) analysis as a discrete chapter, this report adopts an integrated compliance approach. The analysis of how specific design choices satisfy the Functional Requirements (FR) and Performance Requirements (PR) is integrated directly into other chapters.

This methodology ensures that technical evaluations are contextually grounded within the relevant subsystems. To maintain traceability and provide a consolidated overview of system validation, a comprehensive Requirements Verification Matrix is provided in Appendix A, which serves as a cross-reference for all verified criteria discussed throughout the main body of this document.

9 Project Plan

9.1 Overview

This section provides a comprehensive roadmap for the entire S2 Robotic Systems Design Project. The plan is designed using standard project management methods to ensure clarity, accountability, and traceability across all project phases.

9.2 Work Packages Structure

The project is made up of six Work Packages (WPs). Each work package represents a major functional subsystem or a distinct developmental phase. Here is the breakdown of the work structure:

- **WP1 Hardware Design:** Focusing on assembly and combination of all hardware components.
- **WP2 Manipulation:** Focusing on arm control, kinematics, and basic grasping.
- **WP3 Perception:** Focusing on the vision system and object detection.
- **WP4 Navigation:** Focusing on robot base control, mapping, localization, and path planning.
- **WP5 System Integration:** Focusing on communication architecture and software design, and linking all subsystems to work together as a single robot.
- **WP6 System Testing:** Focusing on final validation tests, confirming that the robot meets all product requirements.

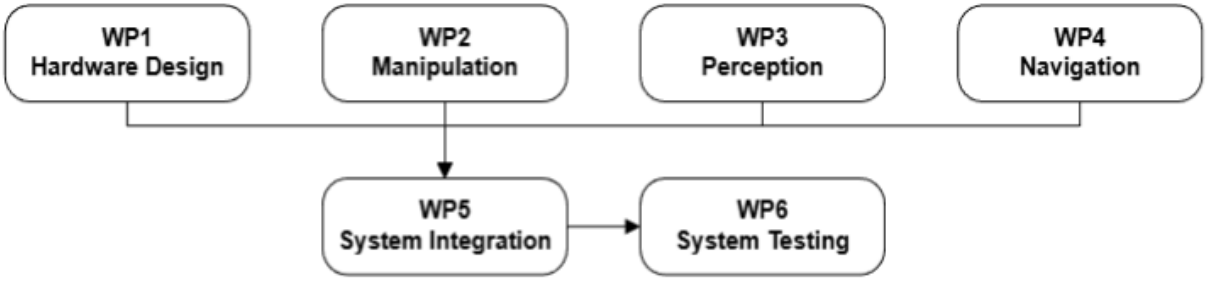


Fig. 26. Work Packages Connections

9.3 Project Gantt Chart

The Gantt Chart illustrates the timeline of the project by defining the Start, Duration, and End for all tasks, and ensures logical sequencing of project development. The full chart is presented on the subsequent page to ensure optimal legibility and clarity.

9.4 Milestones, Deliverables, and Resource Allocation

Task progress is marked by Milestones, which need Deliverables, such as code and reports, to prove acceptance of major phases. Additionally, Resource Allocation appoints clear accountability to a responsible team member, the Assignee, to ensure efficient and effective delivery. The full table is presented on the subsequent page to ensure optimal legibility and clarity.

9.5 Status Traffic-Light System

The Traffic-Light System (TLS) is deployed in the System Testing phase to present a summary of the robot system's fulfillment of requirements. The TLS uses three indicators:

- **Green**: Requirement has been fully Met.
- **Amber**: Requirement is Partially Met (minor issues or deviations exist).
- **Red**: Requirement has been Not Met (major issues or incomplete).

The Project Gantt Chart													
Phase	**Task**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
WP1 Hardware Design													
Concept of Evaluation	Mechanical design												
	Electrical design												
Testing	Hardware validation												
WP2 Manipulation													
Preparation	Arm setup												
Concept of Evaluation	Kinematics solution design												
Implementation	Module implementation												
Testing	Module testing												
WP3 Perception													
Preparation	Camera setup												
Concept of Evaluation	Recognition solution design												
Implementation	Module implementation												
Testing	Module testing												
WP4 Navigation													
Preparation	Lidar setup												
Concept of Evaluation	SLAM solution design												
Implementation	Module implementation												
Testing	Module testing												
WP5 System Integration													
Implementation	Hardware and electrical integration												
	Software communication integration												
	Software module integration												
WP6 System Testing													
Testing	Hardware stability testing												
	Electrical endurance testing												
	Functional requirements testing												
DEMO													
Live closed demonstration													
Open static demonstration													

Milestones and Deliverables				
Milestone	**Due Date**	**Deliverable**	**Description**	**Responsible Person**
WP1	27/02/26	D1.1	Mechanical design file	Junyi Lei
		D1.2	Electrical design file	Junyang Xiao
WP2		D2.1	Manipulation fucntional code	Muhammad Zaman
		D2.2	Manipulation interface code	
		D2.3	Manipulation sucessful backup video	
WP3		D3.1	Datasets and trained model file	Iva Jorgusheska Mingyi Yuan
		D3.2	Perception functional code	
		D3.3	Perception interface code	
		D3.4	Perception successful backup video	
WP4		D4.1	Navigation functional code	Junyi Lei Junyang Xiao
		D4.2	Navigation interface code	
		D4.3	Navigation successful backup video	
Coursework6	20/03/26	D5.1	Final deisgn document	All Team members
WP5	24/04/26	D6.1	Hardware integration video	Junyi Lei Junyang Xiao
		D6.2	Software integration code	Mingyi Yuan Iva Jorgusheska
WP6	01/05/26	D7.1	Hardware stability testing video	Junyi Lei
		D7.2	Electrical endurance testing video	Junyang Xiao
		D7.3	Functional requirements testing report	Iva Jorgusheska
		D7.4	Functional requirements testing successful backup video	Mingyi Yuan
DEMO	08/05/26	D8.1	Promotional video	Muhammad Zaman
		D8.2	Requirements evaluation video	All Team members

Appendices

A Design Requirements Document

A.1 Problem Statement

Autonomous mobile robots equipped with autonomous localization, decision-making, and manipulation are essential for optimizing resource management and enhancing operational efficiency in human-free workspaces. This project is specifically motivated by the need to develop a robust, flexible, and fully integrated autonomous platform that directly addresses difficulties in automated sorting and retrieval tasks.

The project poses challenges that require us to design and integrate an autonomous robotic system capable of reliable object detection, navigation, and manipulation within a constrained and unpredictable environment.

The task requires developing a robot that can detect specific object colours, grasp three objects from the test area, and place them in their corresponding colour-matched storage bins. The robot operates in a netted test environment featuring a flat, hard surface with no steps or uneven terrain.

A.2 Concept of Operations

A.2.1 Mission Overview

The robot will perform autonomous navigation and path planning, detect and avoid obstacles, identify, recognize, and grasp target objects, and retrieve and deposit them into colour-matched bins within a netted test area. The whole mission shall be finished in 20 minutes.

A.2.2 System Actors

Primary System

- **Autonomous Robot:** Mobile platform with sensing, localisation, detection, and object manipulation capability.

External Elements

- **Test Environment:** The test environment is a walled arena featuring a flat, hard floor with no stairs or significant elevation changes. It contains various static obstacles and target objects arranged in a non-fixed configuration. This layout ensures the robot's autonomous navigation and detection capabilities are tested against a unique, randomized environment.
- **Coloured Target Objects and Boxes:** Small different shape, different coloured objects (≤10 cm diameter, ≤300 g weight) randomly distributed within the environment.

A.2.3 Mission Phase

Phase 1 Setup and Mission Start

The operator positions the robot in the designated starting area and powers it on. The system performs a self-check to verify that all sensors and actuators are functional. Once cleared, the 20-minute mission timer begins. The robot then initializes localization and constructs a baseline environmental model from its initial sensor readings to start autonomous operation.

Phase 2 Exploration and Navigation

The robot uses SLAM to autonomously explore and map the environment while avoiding obstacles. It utilizes real-time sensor data for navigation and dynamic obstacle avoidance. If the path is blocked, the robot automatically performs recovery behaviours to reverse and replan a new route.

Phase 3 Object Detection

The robot scans for coloured target objects and bins during exploration using a YOLOv8 model in real-time. Once a target is classified, its relative position is transformed into global coordinates via the TF (Transform) tree and logged into a mission database. If the inference confidence is low, the robot performs a verification maneuver to capture additional frames, ensuring the spatial data is accurate for downstream tasks like grasping.

Phase 4 Approach and Pickup

The robot navigates toward the object while maintaining obstacle clearance. Once in position, it aligns its manipulator and attempts to grasp the object. If a grasping attempt fails or the object is dropped, the robot will retry the pickup until the predefined number of attempts is achieved. If the failure persists, the robot defers the task and proceeds with the remaining targets to ensure mission efficiency.

Phase 5 Return and Deposit

After collecting an object, the robot navigates back to the colour-matched bin and deposits the object. If the deposit fails (e.g., object not released correctly or misses the bin), the robot repositions and attempts a second placement before proceeding.

Phase 6 Mission Continuation

The robot repeats exploration, detection, grasp, and deposit cycles until all three objects have been collected or the 20-minute time limit expires. Throughout the mission, it monitors its battery level and system health. If low power or major sensor issues are detected, it will safely return to the starting area and terminate the mission.

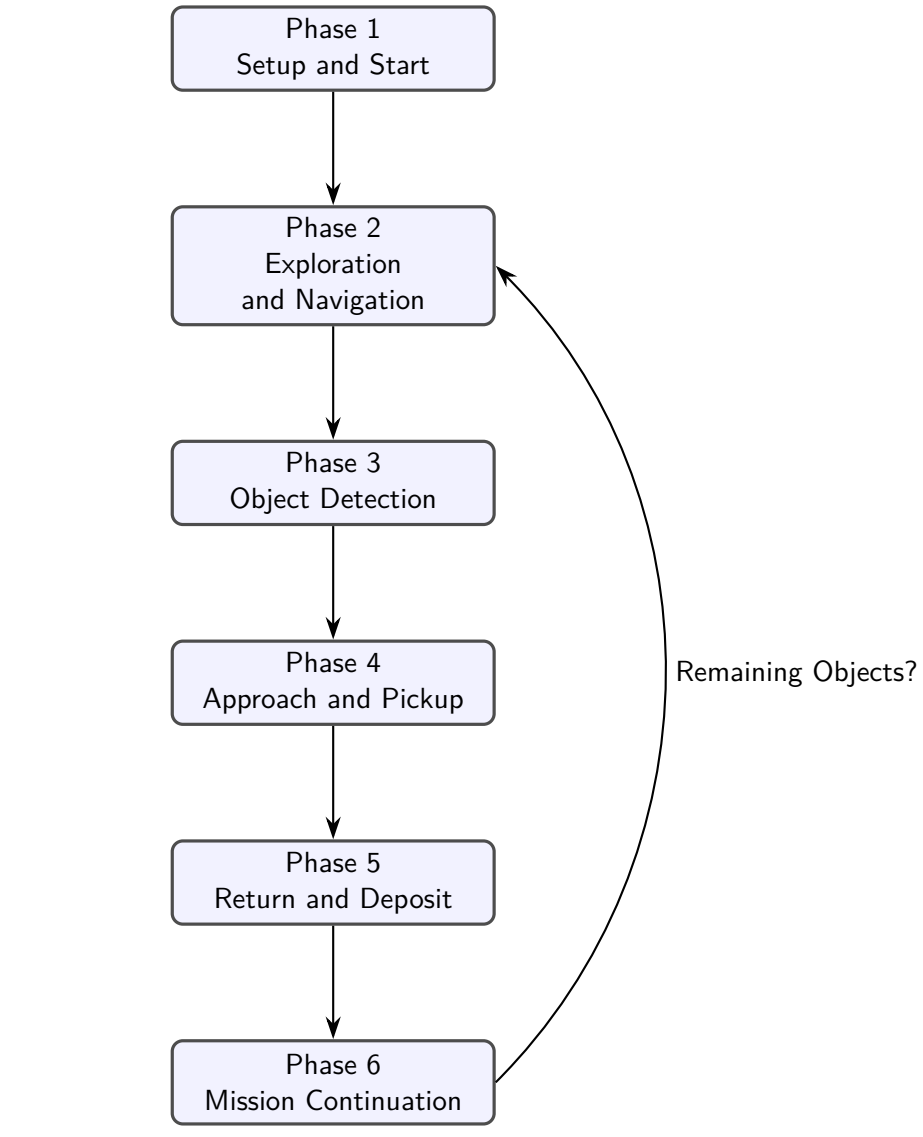


Fig. 27. Mission Phase Flow Diagram for the Autonomous Retrieval Robot.

A.2.4 Assumptions

- **A1:** The test environment features a perfectly flat, hard, and non-deformable floor, and is entirely free of stairs, steps, or significant elevation changes.
- **A2:** The test environment will be strictly indoors and well-lit, with illumination levels maintained between 200 lux and 800 lux.
- **A3:** The target objects, colored boxes, and obstacles are static and randomly placed in the test environment.

A.2.5 Constraints

- **C1:** The system design is constrained to use the hardware provided, including the Leo Rover base, the specific manipulator arm, LiDAR, camera, and the provided power bank.
- **C2:** The physical properties (size, shape, weight, color) of the target objects, colored boxes, and obstacles are strictly defined and cannot be modified.

A.3 Functional and Performance Requirements

A.3.1 General Mission

FR1: The robot shall autonomously perform the full retrieve-and-deposit mission after an operator-initiated start command.

PR1.1: The robot shall complete the entire mission with zero human intervention.

PR1.2: The robot shall complete the entire mission within a maximum time limit of 20 minutes.

FR2: The robot shall provide continuous and visible status feedback throughout the entire mission.

FR3: The robot shall incorporate a modular hardware design to facilitate the rapid replacement of key components.

A.3.2 Navigation

FR4: The robot shall map, self-localise, and navigate without hitting obstacles.

PR4.1: The mapping accuracy shall be less than 10 cm.

PR4.2: The maximum localisation error from the real-world position shall be 10 cm.

FR5: The robot shall actively detect environmental obstacles while navigating.

PR5.1: The system shall detect obstacles with a minimum width of 20 cm within the LiDAR effective scanning range.

PR5.2: The system shall detect obstacles within a radial range of 30 cm to 150 cm.

PR5.3: The system shall detect obstacles maintaining a minimum 5 cm safety clearance.

FR6: The robot shall position its base such that target objects and bins are within the manipulator's reachable workspace.

FR7: The robot shall localise the target object and autonomously approach it.

PR7.1: The robot shall halt at a range of 30 cm to 35 cm in front of the target object.

PR7.2: The maximum object positioning error shall be 5 cm.

PR7.3: The success rate of navigating to the target object shall be at least 80%.

FR8: The robot shall localise the target bin and autonomously approach it.

PR8.1: The robot shall stop its base at a distance between 30 cm and 35 cm from the target bin.

PR8.2: The success rate of navigating to the target box shall be at least 80%.

A.3.3 Detection

FR9: The robot shall detect three colour classes (Red, Yellow, and Purple) objects from the provided object set.

PR9.1: The robot shall detect objects within a range of 20 cm to 60 cm.

PR9.2: The object detection success rate shall be over 80%.

FR10: The robot shall detect three colours (Red, Yellow, and Purple) provided bins.

PR10.1: The robot shall detect bins at a physical distance within a range of 20 cm to 60 cm.

PR10.2: The bin detection success rate shall be over 80%.

A.3.4 Grasping

FR11: The robot shall grasp target objects located within its reachable workspace (within a range of 5 cm to 30 cm).

PR11.1: The grasping success rate shall be over 80%.

FR12: The robot shall automatically reattempt if grasping failed.

PR12.1: The reattempts shall be up to a maximum of five times.

FR13: The robot shall securely hold target objects during both stationary and mobile operations.

FR14: The robot shall deposit the target object into its colour-matched bin (the robot is assigned to bins based on logic rather than real-time bin colour recognition).

PR14.1: The deposit success rate shall be over 80%.

A.3.5 Electrical Communication

FR15: The power system shall provide continuous and regulated power to all onboard systems for the entire mission duration.

PR15.1: The power system shall support a 20-minute demonstration plus a 10-minute reserve, ensuring a minimum total operating time of 30 minutes.

FR16: The communication link in this robotic system shall maintain high reliability and low latency.

PR16.1: The data packet loss rate shall be less than 3%.

PR16.2: One-way communication latency for command processing shall not exceed 500 milliseconds.

A.3.6 Safety

FR17: The locomotion control system shall enforce kinematic limits on the robot to prevent hazardous operations.

PR17.1: Maximum angular speed shall be at $60°/s$.

PR17.2: Maximum linear speed shall be at 0.4 m/s.

FR18: The complete structural design shall prevent physical injury to human operators and comply with manual lifting limitations.

PR18.1: The total weight of the robot, including all payloads and structural attachments, shall definitely not exceed 18 kg.

FR19: The electrical systems shall feature comprehensive electrical shielding and isolation.

PR19.1: Exposed electrical leads and terminals shall be covered to prevent accidental shorting.

FR20: The robotic system shall incorporate redundant emergency shutdown mechanisms.

PR20.1: The robotic system shall include a physical power-off switch.

FR21: The robotic system shall implement a remote software stop command.

PR21.1: The robot shall reach a complete standstill within 1.0 seconds of receiving a halt command.

A.4 Requirements Verification Matrix

Table 5. Requirements Verification Matrix

Requirement	Verification Success Criteria	Verification Method
FR1	The robot begins the mission sequence and enters ‘INIT‘ state upon receiving the start command.	1. Send the start command via Terminal. 2. Observe the robot initiate and perform the full mission.
PR1.1	3 out of 3 mission runs are completed without any manual teleoperation, resets, or physical assistance.	1. Start the program 3 times in the test arena. 2. Monitor and record any required manual interventions.
PR1.2	The calculated average duration of 3 full mission runs is ≤ 20 minutes.	1. Use system timestamps. 2. Time 3 full mission runs and calculate the average duration.
FR2	Terminal logs and visual indicators consistently display the current operational state.	1. Monitor terminal logs and visual indicators during a full mission run.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
FR3	Major subsystems can be completely removed and reattached within 15 minutes using standard hand tools.	1. Perform a timed teardown and reassembly of major subsystems using standard tools.
FR4	The robot successfully reaches all designated goal poses with zero physical collisions with any obstacles.	1. Set goal poses in the navigation system. 2. Monitor physical robot behavior for 0 collisions.
PR4.1	All measured physical distances correspond to the generated map dimensions with an error of < 10 cm.	1. Generate a map of a known area. 2. Measure physical distances between landmarks and compare to map.
PR4.2	The robot's estimated pose in the navigation stack matches the physical ground truth within a 10 cm radius.	1. Command robot to specific coordinates. 2. Physically measure the actual position against expected.
FR5	Unmapped dynamic obstacles instantly appear in the local costmap and trigger a path replanning event.	1. Place unmapped obstacles in the robot's planned path. 2. Verify detection via sensor data visualization.
PR5.1	20 cm wide test cylinders consistently generate valid laser scan returns and are marked in the occupancy grid.	1. Place 20 cm wide test cylinders in the FOV. 2. Verify valid laser scan returns and map updates.
PR5.2	Targets placed at 30 cm, 100 cm, and 150 cm are successfully detected and actively avoided during navigation.	1. Place targets at 30 cm, 100 cm, and 150 cm. 2. Verify targets are detected and utilized for avoidance.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
PR5.3	The physical footprint of the robot maintains a minimum distance of ≥ 5 cm from all detected obstacles.	1. Configure inflation radius. 2. Run navigation tests and measure the closest physical approach.
FR6	The final navigation pose allows the end-effector to physically reach the target without triggering a singularity.	1. Send robot to target zone. 2. Verify final base position allows the arm to reach target without singularity.
FR7	The navigation stack successfully generates and executes a trajectory towards the detected target object.	1. Place target object in view. 2. Verify generation of approach commands towards the object.
PR7.1	The final measured distance between the robot's front reference and the object is between 30-35 cm.	1. Allow robot to perform approach. 2. Physically measure the final distance. Repeat 5 times.
PR7.2	The lateral offset between the gripper's center and the target object is ≤ 5 cm after halting.	1. Measure the lateral offset (error) of the object relative to the expected position after halting.
PR7.3	At least 8 out of 10 approach trials result in the robot stopping within distance and lateral offset constraints.	1. Conduct 10 approach trials from varying starting points. 2. Verify at least 8 successes.
FR8	The navigation stack successfully generates and executes a trajectory towards the target bin.	1. Place target bin in view. 2. Verify generation of approach commands towards the bin.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
PR8.1	The final measured distance between the robot base and the target bin is between 30-35 cm.	1. Allow robot to perform bin approach. 2. Measure final distance from base to bin. Repeat 5 times.
PR8.2	At least 8 out of 10 bin approach trials are completed successfully within constraints.	1. Conduct 10 approach trials towards the bin. 2. Verify at least 8 successes.
FR9	The vision node publishes the correct semantic label (Red, Yellow, or Purple) matching the physical object.	1. Present the Red, Yellow, and Purple objects to the camera. 2. Verify correct classification labels.
PR9.1	The vision system publishes valid bounding boxes and coordinate data for objects placed at any distance within the 20-60 cm range.	1. Place objects at 20 cm, 40 cm, and 60 cm marks. 2. Verify the detection node publishes valid bounding data at each distance.
PR9.2	At least 8 out of 10 detection trials correctly identify and localize the object under varying angles.	1. Conduct 10 detection trials per object with varied angles. 2. Verify at least 8 successful detections.
FR10	The vision node publishes the correct semantic label (Red, Yellow, or Purple) matching the physical bin.	1. Place bins in FOV. 2. Verify the vision system detects them correctly.
PR10.1	The vision system publishes valid bounding boxes and coordinate data for the bins placed at any distance within the 20-60 cm range.	1. Place bins at 20 cm, 40 cm, and 60 cm marks. 2. Verify the detection node publishes valid bounding data at each distance.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
PR10.2	At least 8 out of 10 detection trials successfully identify the bin under operational conditions.	1. Conduct 10 detection trials for the bins. 2. Verify at least 8 successful detections.
FR11	The robot end-effector successfully grasps, secures, and lifts target objects located within its reachable workspace (5-30 cm range).	1. Place target objects at the 10 cm, 20 cm, and 30 cm marks. 2. Command the grasp sequence and verify the object is secured and lifted off the surface.
PR11.1	At least 8 out of 10 automated grasping cycles result in a secure hold and lift.	1. Perform 10 automated grasping cycles. 2. Verify at least 8 successful lifts.
FR12	The system automatically detects a missed grasp (empty gripper) and triggers a reset-and-retry sequence.	1. Intentionally move the object slightly out of reach during grasp. 2. Verify automatic retry sequence.
PR12.1	The system logs a failure and aborts the specific object task after exactly 5 consecutive unsuccessful attempts.	1. Remove object after initial detection. 2. Count retry cycles and verify system aborts after 5 attempts.
FR13	The grasped object is not dropped during any arm sweeping motions or while the base is navigating.	1. Grasp object. 2. Perform arm sweeping motions and drive base. 3. Verify object is retained.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
FR14	The robot plans a path to the correct color-matched bin and successfully opens the gripper to release the object.	1. Grasp a specific colored object. 2. Verify robot plans trajectory to corresponding bin and releases it.
PR14.1	At least 8 out of 10 deposited objects land fully inside their respective designated bins.	1. Run 10 full retrieve-and-deposit cycles. 2. Verify at least 8 successful drops inside the box.
FR15	No subsystem experiences brownouts or unexpected resets during high-load simultaneous tasks.	1. Monitor system voltage logs during a high-load simultaneous driving and arm movement task.
PR15.1	The battery voltage remains above the critical safe discharge threshold after 30 minutes of continuous operation.	1. Run robot continuously under load for 30 minutes. 2. Measure end voltage stays above critical threshold.
FR16	Telemetry and commands transmit continuously without connection drops or noticeable stuttering.	1. Maintain a constant ping from base to robot while streaming sensor data.
PR16.1	Network diagnostic tools confirm a packet loss rate of strictly $< 3\%$ over a sustained 5-minute test.	1. Run network stress test for 5 minutes during movement. 2. Verify packet loss statistics.
PR16.2	The measured time delay between issuing a command and the actuator initiating movement is < 500 ms.	1. Send movement command via remote station. 2. Measure delay until action begins is < 500 ms.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
FR17	The control software explicitly clamps velocity commands so physical limits are never exceeded.	1. Publish velocity commands exceeding limits. 2. Verify the executed speeds are clamped.
PR17.1	Peak angular velocity data from odometry logs never exceeds 1.05 rad/s ($60°/s$).	1. Log odometry during max rotation commands. 2. Analyze peak angular velocity is $\leq 60°/s$ (1.05 rad/s).
PR17.2	Peak linear velocity data from odometry logs never exceeds 0.4 m/s.	1. Log odometry during max forward commands. 2. Analyze peak linear velocity is ≤ 0.4 m/s.
FR18	The chassis presents no exposed sharp edges and allows safe manual lifting by operators.	1. Conduct physical inspection for sharp edges. 2. Perform test lift with operators.
PR18.1	The fully assembled and operational robot registers exactly ≤ 18.0 kg on a calibrated scale.	1. Weigh the fully assembled robot on a calibrated industrial scale.
FR19	Control logic circuits are electrically isolated from high-current paths, and wiring is shielded.	1. Visual inspection of layouts, optoisolators, and wire routing.
PR19.1	All connections, battery terminals, and splices are protected with heat shrink or physical enclosures.	1. Visual inspection of the entire wiring harness and heat shrink tubing.
FR20	Both the hardware switch and the software kill command function independently to stop the robot.	1. Verify presence of both physical and software independent E-stops.
PR20.1	Toggling the physical E-stop switch immediately cuts power to all drive and manipulator motors.	1. Engage physical switch while moving. 2. Verify immediate loss of motive power.

Continued on next page

Requirement	Verification Success Criteria	Verification Method
FR21	Activating the specific override command successfully halts all execution loops and commands zero velocity.	1. Trigger the software E-stop via the control station while navigating.
PR21.1	All base and arm motion stops completely within ≤ 1.0 seconds of the software E-stop being triggered.	1. Trigger E-stop during max speed. 2. Measure time until wheel encoders read zero is ≤ 1.0 s.

B Workplace Charter

B.1 Statement of Purpose

This Workplace Charter outlines the agreed framework for how our team will operate during the ***AERO62520 Robotic Systems Design Project***. It defines the team's shared expectations regarding communication, collaboration, conduct, and conflict management. It will serve as the basis for maintaining accountability, resolving disagreements, and ensuring that all members work respectfully and effectively toward common goals. In the event of uncertainty or conflict, this Charter will provide clear procedures and standards to support a fair and transparent resolution.

This Charter is a living document and may be reviewed and updated by mutual agreement if project circumstances, feedback, or team needs change. Any modifications must be discussed openly, approved by the majority, and documented by updating this document.

B.2 Principles and Commitments

Our team is dedicated to creating a professional, respectful, and inclusive environment that fosters learning, innovation, and overall well-being.

- **Respect and Inclusion:** Zero tolerance for any kind of discrimination. Every member is valued for their unique background and has the right to express their perspective in a safe and respectful environment.
- **Transparency and Trust:** All relevant information should be shared openly on the team's official channels. Core decisions must be documented and justified. Team members have the responsibility to answer other members' project questions promptly and fully.
- **Accountability and Quality:** Each team member will take ownership of their assigned tasks and deliver work to the best of their abilities, maintaining a high professional standard while prioritising reliability, accuracy, and timely delivery.
- **Collaboration, Learning, and Continuous Improvement:** All team members are expected to continuously develop their skills, share knowledge, and remain open to learning throughout the project. Constructive feedback is encouraged at all stages and must be given respectfully and received with professionalism and openness.
- **Academic Integrity and Data Compliance:** All work must be original, and any personal or private information should be handled carefully. Academic dishonesty, including plagiarism

and data fabrication, is strictly prohibited. Data sharing outside the team requires approval from all the members.

- **Health and Safety:** Members must follow all the health and safety rules of the university, and the lab, and report any safety or well-being issues immediately.

B.3 Team Rules

To ensure collaboration, productivity, and mutual respect within our team, the following rules provide guidelines for our work practices and communication throughout the project.

B.3.1 Working Hours and Availability

1. **Core working days:** Monday to Saturday. No expectation of routine working activity on Sunday, which is designated as a shared rest day to align with typical UK weekend schedules.
2. **Additional effort:** Each member is expected to contribute approximately 5 hours per week outside of scheduled lab sessions for simulations, debugging, and other project work.
3. **Availability window:** Members should be reachable and respond to project-related communications and attend meetings within the working window of 10:00–19:00 on core working days (Mon–Fri).
4. Any additional time commitments should be discussed and agreed upon by all team members in advance.

B.3.2 Attendance and Punctuality

1. **Meetings and labs:** Members must attend all scheduled team meetings and assigned laboratory sessions. If unable to attend, notify the team at least 24 hours in advance whenever possible.
2. More than two unexcused absences or repeated lateness (15+ minutes) without prior notice will initiate the conflict resolution process.

B.3.3 Language and Conduct

All communication must be in English and remain professional, inclusive, and respectful. Aggressive or discriminatory behaviour will not be tolerated. Constructive feedback is encouraged when

offered respectfully and with the goal of helping the team.

B.3.4 Modes of Communication and Expectations

1. **Formal communications (decisions, approvals, meeting minutes, or supervisor correspondence):** All major decisions must be communicated via email and acknowledged by recipients.
2. **Day-to-day coordination, file sharing, and discussions:** All relevant files, documentation and updates should be posted on the Teams channel.
3. **Informal or quick queries:** WhatsApp or similar platforms may be used for short, non-binding messages. Formal requests should never be made solely through informal channels.
4. **Response times:** Members should aim to respond within the same working day (Mon–Sat) for non-urgent messages where practical, and within 24 hours on working days for routine matters. Messages sent on Sundays or late at night may be answered on the next working day. During urgent deadlines, faster response times may be requested by the team.

B.3.5 Decision Making

Decisions will be made through open discussions and majority votes. Disagreements should be supported with reasoning, evidence, or research to promote constructive dialogue and sound outcomes.

B.3.6 Work Quality and Deadlines

If a member anticipates difficulty meeting a deadline or completing a task, they must inform the team promptly. This allows for timely support or task reallocation to maintain project progress. While the team values mutual support, each member is responsible for proactive communication and contributing their fair share to the workload.

B.3.7 Conflict Resolution

The conflict resolution process is outlined in the flowchart. Team members must first attempt to resolve conflicts through direct and respectful discussion. If the issue persists, it should then be escalated according to the procedure detailed in the flowchart.

B.4 Daily Activity Conflict Avoidance

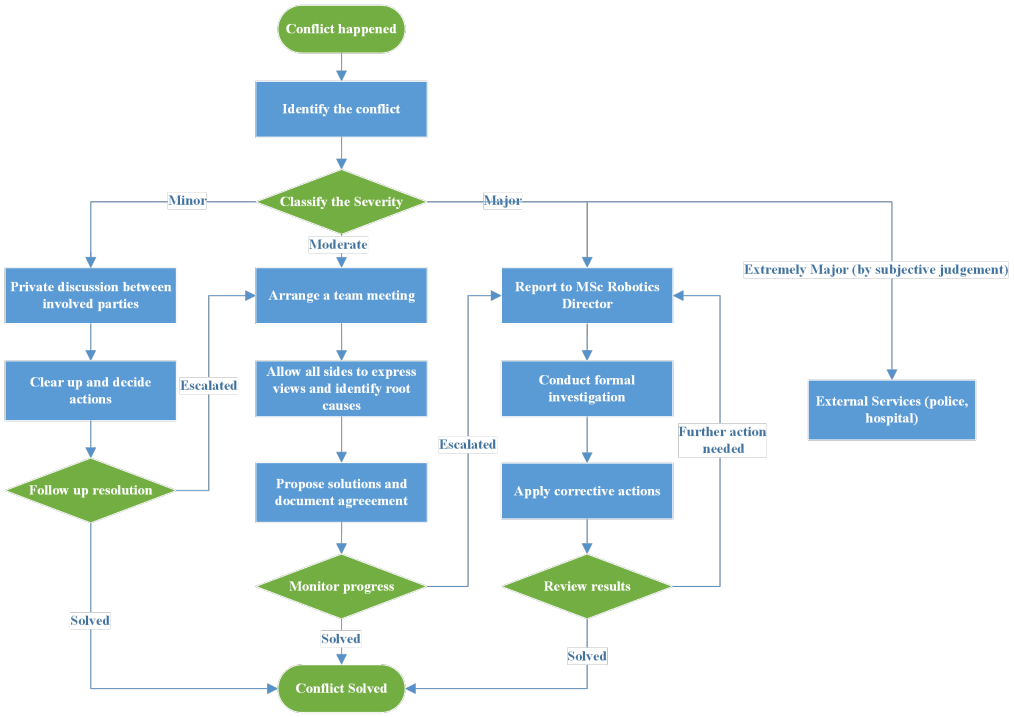
- **Clear Ownership:** Each subtask will have one clearly assigned owner to avoid overlap or confusion. Task assignments will be based on members' skills and demand, with mutual agreement during planning discussions.
- **Weekly Progress Sharing:** Each team member will provide a brief update to the group each week, summarising the progress made on their assigned tasks. These updates ensure that all members have a clear understanding of the current project status, upcoming work, and any challenges that may require support.
- **Data Management:** All project materials should be kept on GitHub or OneDrive, avoiding confusion from multiple channels.
- **Respectful Communication and Support:** When disagreements or concerns arise, members should raise them calmly and privately at first, focusing on resolving the issue, not assigning blame. Team members are encouraged to check in with one another, offer support, and ask for help early to avoid stress buildup.
- **Balanced Workload:** Members are encouraged to plan their individual working time outside scheduled lab sessions according to their assigned sub-projects and study commitments. The team will monitor workload regularly to ensure fairness and sustainability.
- **Work–Life Balance:** The team recognises the importance of maintaining a healthy balance between project responsibilities and personal well-being. Members are not expected to work excessive hours, and rest days should be respected. Breaks, holidays, and personal time should be used appropriately, and the team will support adjustments when needed to prevent burnout or undue pressure.

B.5 Conflict Resolution Flowchart

B.5.1 Classification of Conflicts

- **Minor:** Simple misunderstandings or small disagreements that do not impact work or relationships.
- **Moderate:** Disagreements that begin to affect collaboration or productivity but can be resolved through discussion or mediation.
- **Major:** Serious disputes that disrupt teamwork or involve ethical or academic issues requiring intervention by the supervisor or university officials.

B.5.2 Conflict Resolution Flowchart



B.5.3 Supporting Pages for Serious Conflicts

The following resources are linked to the Major Conflict stage of the flowchart. These resources provide guidance, reporting options, and official university procedures for resolving serious or sensitive issues.

https://www.manchester.ac.uk/connect/jobs/equality-diversity-inclusion/

https://www.manchester.ac.uk/connect/jobs/equality-diversity-inclusion/initiatives/

https://www.regulations.manchester.ac.uk/academic/student-charter/

https://www.studentsupport.manchester.ac.uk/taking-care/

https://www.reportandsupport.manchester.ac.uk/